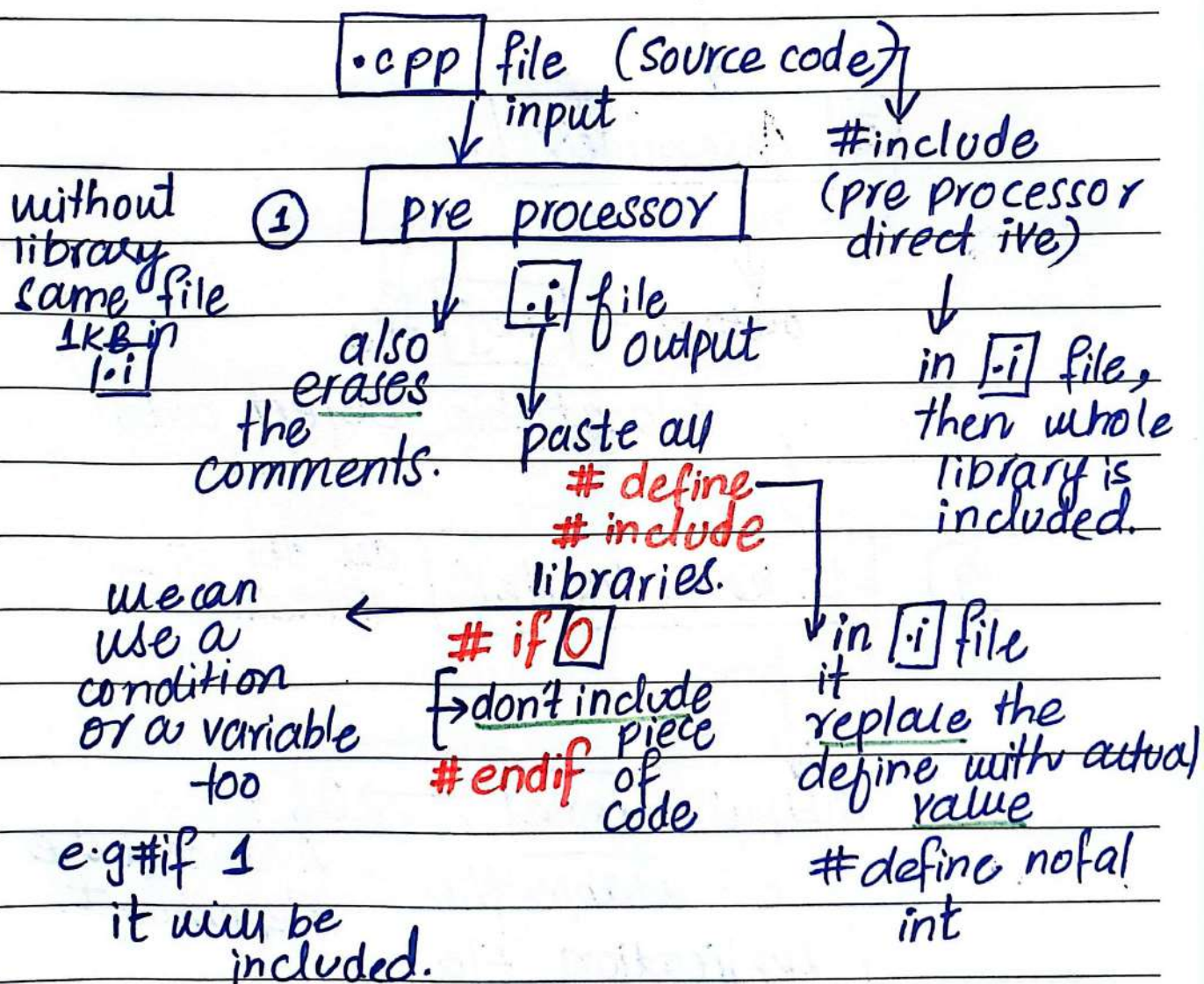


Fatima Imran 22L-6946

Compiler Construction

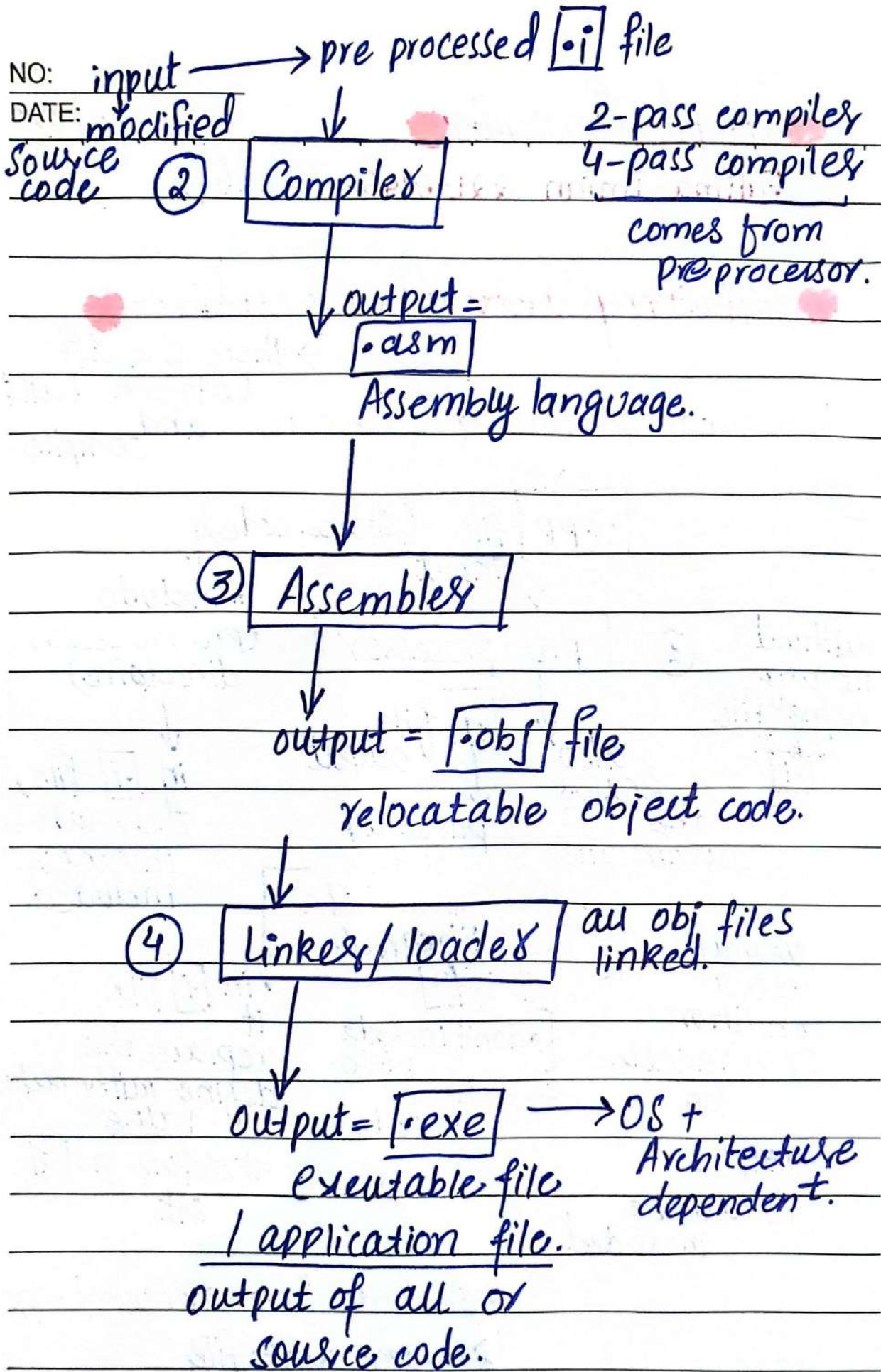
→ there is a diff between build and compile.

Language processing phase :-



#include "close Bracket.cpp"

→ replace with file



NO: _____

DATE: _____

→ if there are two files in a project and one file has error. Whole project after build gives an error due to linker/loader.

→ function redefinition error

obj >.i (replace code)

if used
↑ still
error

so #include "closeBracket.cpp"
int multiply (int a, int b); //prototype

if same
func in
diff files

↓
linker error

if same
func in
same file

↓
compile error

fatal error → due to linker

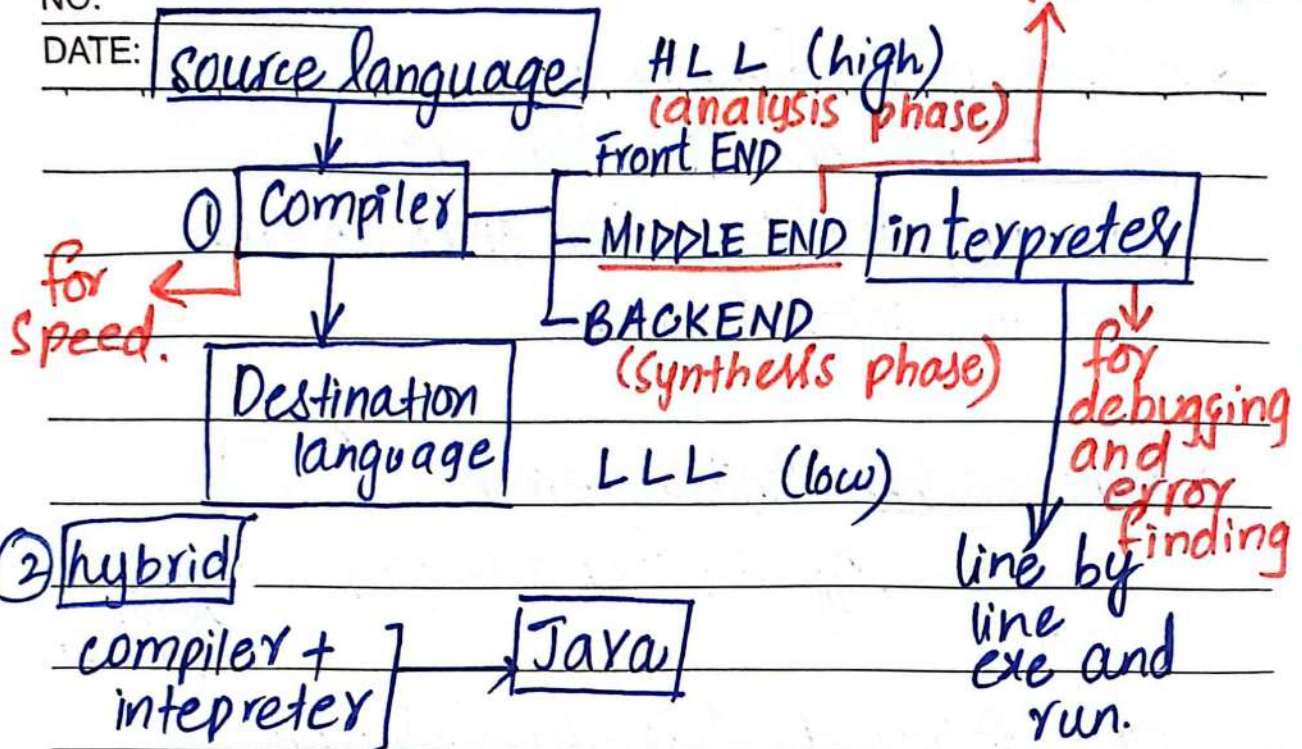
function defined (prototype)
but not actually present.

1.1, 1.2

NO:

DATE:

for optimization



② hybrid

compiler + interpreter

Java

Source code → byte code

input

JVM (Java virtual machine)

• i

PD (program dependent)

FE

L1 dependent

↓ IR (intermediate code) → program independent

↓ IR

ME

machine independent

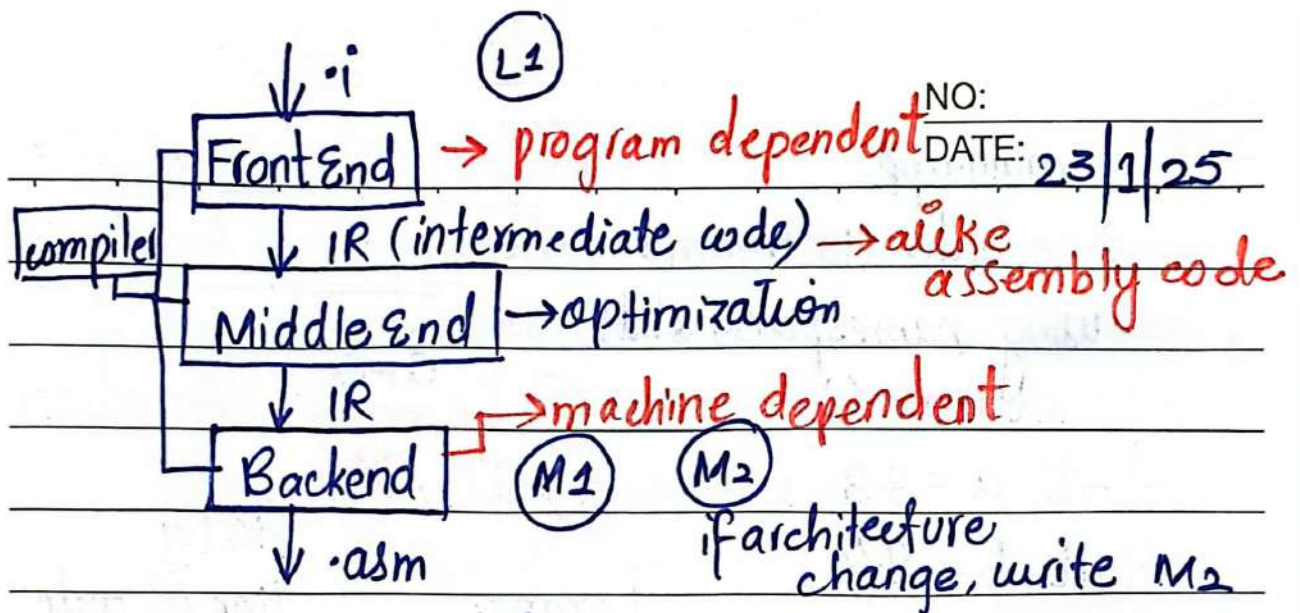
↓ Optimized IR

machine code / asm.

③ Just-in-time compiler

M1, M2, M3
mac linux OS

compile machine independent to machine dependent code for time and efficiency.



e.g if we

have a new language L_2 , do we have to write another $M_1, M_2, \dots$ **No** because $L_2 \rightarrow IR$ and can be run on multiple backends.

→ so a single representation (L) is enough and machine can run it.

⇒ L_1 & L_2 are IR (intermediate representation)

Topic: Front End Phases:

NO:

DATE:

main.cpp

① Lexical Analysis:

② Syntax Analysis:

CFG

③ Semantic Analysis

④ IR, 3AC

Firstly line of code comes in

```
#include <iostream>
using namespace std;
int main()
```

```
float a = 3.3 ;
```

```
float b = 4.4 ;
```

```
float c ;
```

if removed syntax ~~C = a + b * 60 ; !~~

error

identify word in C++

Lexical Analysis or Scanning phase

obj: to tokenize your input

DFA

Frontend

Specification:

RE/FA

Error Handling

Lexeme (lexeme)

file is input

input buffer

2 buffers.

C = |a| + |b| * |6| |0| ;
then this stream of char line goes to LA char by char

output of LA:

stream of tokens.

<class, pos>

<id, 1> <=>

<id, 2> <+> <id, 3> <*>

<60> < ;>

identifier

Lexical error:

like My Ma

Symbol Table:

pos	letter
1	c
2	a
3	b
4	

* operators & numbers don't go to table.

Lexical Error:

std
\$td x

NO:

DATE:

cuz \$ doesnot belong to
any class alphabet set

→ char that doesnot exist
in language so,

1num is not a lexical
error.

alphabet set
identifier:

a-z

A-Z

_,

63
alphabets.

→ position fetch.
Rel btw LA and symbol table =
Read and write

$c = a + b * c$

$\langle id, 1 \rangle \langle = \rangle \langle id, 2 \rangle \langle + \rangle$

$\langle id, 3 \rangle \langle * \rangle \langle id, 1 \rangle \langle ; \rangle$

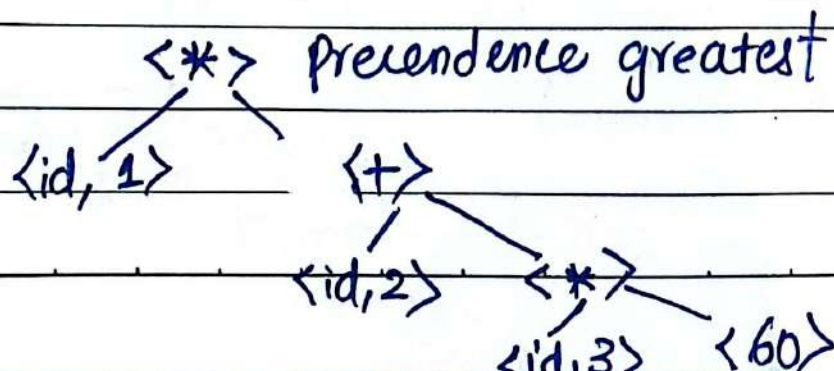
There is only
one symbol table,
redefinition error comes
due to it.

→ relate to symbol table

② Syntax Analysis: CFG, PDA

→ also error handling

↓
Output: Parse Tree:



NO:

DATE:

num_1 X

need some kind of operator.

input: Parsetree

③ Semantic Analysis:

output:

Annotated PT

attributed grammar

scope,
type mismatch,
not declared.

errors:

int a = "a";

→ type mismatch

→ redefinition error

→ array index

5.5 error.

→ declaration

Coersion:

c = a * b + 60;

is an integer and all others are float so warning.

check, convert 60 → 60.0

Compiler

NO:

DATE: 4/2/25

(Analysis Phase)

Frontend

↓ program dependent

→ Lexical Analysis

→ Syntax Analysis

→ Semantic Analysis

→ Intermediate representation (IR)

→ machine independent

Middleend → Optimization

(Synthesis Phase)

Backend

↓ machine dependent

→ Code Generator

→ code Optimization.

→ to reduce # of instructions

3AC

three address code

→ input & output is → (no more than 3 mem location and 1 operation)

Lexical Analysis :-

2 input buffers
(stream of ch)

output :-

Stream of tokens.

file

4096 bytes

Plols=inli+ cildx60

DFA

input only in characters.

result of FA

Token

<class, pos>

Symbol Table

1	Pos
---	-----

lexeme begin

forward

btw this we know lexeme

(meaningful word)
formed → it got accepted in DFA.

NO:

DATE:

e.g keyword = else

3.1-3.4



NFA

if reserve word / keyword
fi identifier

↓ ideally
in symbol
table

crosscheck
from

Symbol
table

→ no tokenization on
basis of spaces.

↓ in form of
linked list,
tree,

a new, graph,

hashmap

1num → not a lexical
error.

↓ ideal time

AVL Tree → balanced

O(1)
for read/write

Symbol Table → first things that are
variable in nature

→ function names.

→ # of parameters

→ parameter type

pass by copy?

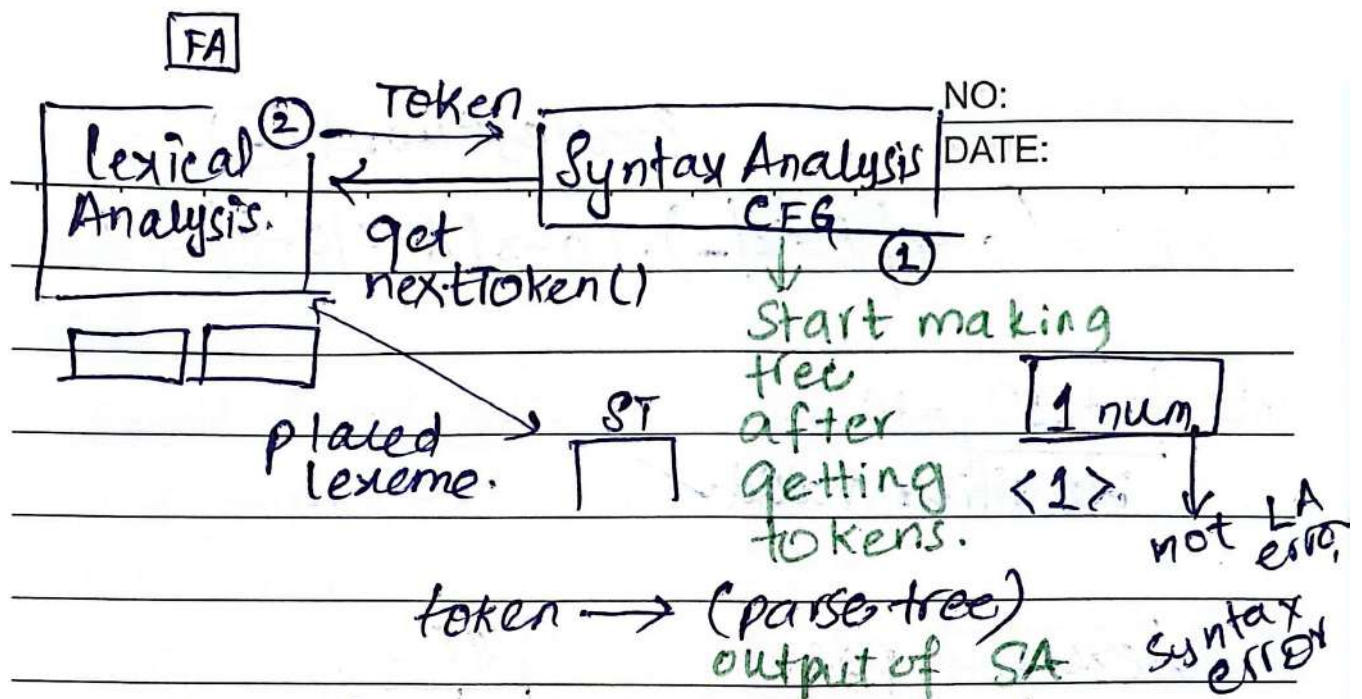
“ “ reference?

Compiler → 1st phase → LA

Compilation of code

intelligence
↓ C/C++ before
compiling.

first phase → Syntax Analysis.



so, lexical error $\rightarrow$ not a part of language e.g. a char or anything

$\rightarrow$ directly token, things which are not written in symbol table.

$\rightarrow$ modify specification of tokens in runtime $\rightarrow$ separate phases of LA & SA?
LA don't directly form tokens

LA $\rightarrow$ first phase is scanning
 $\downarrow$ (remove comments)
 tell which line error, line no. $\downarrow$ White space. Spaces, newline

NO:

DATE:

Smallest word. (-)

$$RE = (a-z/A-Z/-) \cdot (a-z/A-Z/0-9/-)^*$$

int int_10 = 10

identifiers valid ✓

lexeme

pos

ST

token

<id, 1>

1	pos
---	-----

2	ini
---	-----

3	etc
---	-----

} lexeme.

<id, 1> <= > <id, 2> <+ >

<id, 3> <* > <60 >

(tokens)

fill | 0 | alt | | p | 0 | s | | n

can't tell
'pos' is float

NO: 6 Feb 2025

DATE:

float pos;

float initial;

float rate;

pos = initial + rate * 60;

String course = "CC"

|||| ... |||

lexeme, scope,

Lexical Analysis

global,
local

Symbol Table

have type, position,
mem address

stream of
characters

Lexical Analysis

Token

Syntax
Analysis

[eh]

error
handling

read/
write

[Eh]

get
Next
Token()

ST

read/write

↑ like hash
map

Reserve Words Table

every keyword
have different
functionalities attached,
that's why we need
this table

*without
automata, we
cannot recognize
a lexeme.

e.g
<keyword, 9>

<keyword, pos>

String literal → "I am teaching compiler
string course = construction
course for first time in my life at
FAST-NUCES Lahore"

NO:

DATE:

Literal Table

↓ we only store string literals.

→ Our storage will be compromised due to it that's why we store in literal table.

3 data storage → RW Table → Fixed, already filled, with their respective functionalities.

→ ST

→ LT

< keyword, 9 >

< id, 1 >

< literal, 1 >

uses lex
syntax
sem
3AC
Code Gen

ST	
pos	lexeme

Eh

* error handler
attached to every
phase

all due
to type,
mem locations.
....

→ panic mode → compiler phase

↓ corrections of code

if → if main → main

$$pos = initial + rate * 60$$

Stream of Characters.

NO:

DATE:

Lexical Analysis

→ FA

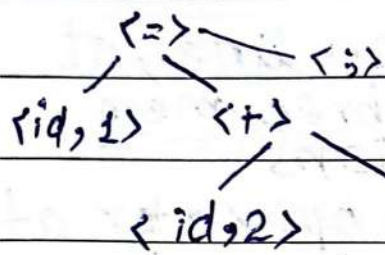
Stream of tokens

$\langle id, 1 \rangle \langle = \rangle \langle id, 2 \rangle \langle + \rangle \langle id, 3 \rangle \langle * \rangle \langle 60 \rangle \langle ; \rangle$

Syntax Analysis

→ CFG, PDA

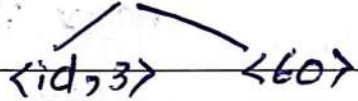
ST



we have
CFG →
declarative
statement
 $\langle key..., 9 \rangle$

pos	lexeme	Type
1	pos	float
2	initial	float
3	rate	float

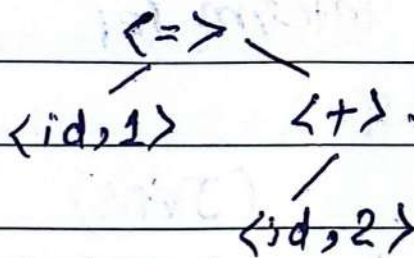
parse tree



we get to know
type from RWT,
we put it in ST.

Semantic Analysis

→ attributed
grammar



annotated
parse tree

$\langle id, 3 \rangle$

int to float

→ Frontend

Intermediate Rep

applied on
tree

$\langle 60 \rangle$

to generate temp code

3AC

$t1 = \text{int to float}(60)$
 $t2 = \langle id, 3 \rangle * t1$
 $t3 = \langle id, 2 \rangle + t2$
 $\langle id, 1 \rangle = \langle t3 \rangle$

// temporary variable

SDD (Syntax
directive definition)

SDD (" " translation)

NO:

DATE:

pg # 7

optional phase

Middle

Optimization

→ # of line decrease

3AC

$t1 = \langle id, 3 \rangle * 60.0$
 $\langle id, 1 \rangle = \langle id, 2 \rangle + t1$

↓

Code Generator

Backend

out is

3AC

LDF R2, L3
(load float)

→ in one line, at max three mem locations

MULF R2, R...

→ one operator at max in each line

assembly language.

input = stream of characters.

↓

Frontend's output → 3AC or interm rep

→ java bytecode

is actually a 3AC, (JVM)
compromise speed to achieve portability.

IR. → JVM → architecture dependent assembly.

11-2-2025 Lexical Analysis

NO:

DATE:

Regular Expression:- $\rightarrow$ string validation.

$L = \{a\}$ $RE = a$

$L = \{if, else\}$ $RE = if + else$

Operators:-

$+$ (choice) $\cdot$ (concatenation)

$*$ $\rightarrow$ have upper bound in machine implementation.

if 10 locations, then 63^{10} variations.

$+$ (superscript)

$\Lambda \rightarrow$ empty initialization.

$()$ for precedence.

identify (id) $\Rightarrow (a+b+c+\dots+z+A+B+\dots+Z+_{-})(a+b+\dots+z+A+B+\dots+Z+0+1+\dots+9+_{-})^*$

$\downarrow$

complexity, no reusability.

Regular Definition:-

$(-)$ tells range

'|' OR

regular expression

{ digits $\rightarrow 0|1|2|\dots|9$ }

{ Small Alphabets $\rightarrow a|b|c|\dots|z$ }

{ Capital Alphabets $\rightarrow A|B|C|\dots|Z$ }

{ underscore $\rightarrow _$ }

$\rightarrow$ di (regular def) & Σ

letters $\rightarrow$ small Alphabets
| Capital Alphabets

NO:

DATE:

letter $\rightarrow$ letters | underscore

id $\rightarrow$ letter $\cdot$ (letter | digits)*

digits $\rightarrow$ 0-9

small Alphabets $\rightarrow$ a-z

capital Alphabets $\rightarrow$ A-Z

underscore $\rightarrow$ _

$\rightarrow$ we can use
hyphen (-)

YourLex $\rightarrow$ keywords | id | numbers |
operators | punctuations

CFG $\rightarrow$ basecase, recursive type.

13 Feb 2025

pg 127

.i

```
int main() { return 0;
```

```
int val = 10;
```

```
float num = 10.0;
```

```
for (; val == 10) {
```

```
    val++; }
```

```
if (val >= 11)
```

```
    printf("Hello");
```

keyword $\rightarrow$ int | main |

float | for | if |

printf

Regular Operator:-

NO: _____

DATE: _____

→ start of line $\Rightarrow \wedge$

$\Rightarrow \wedge \text{int}$ highlight string containing lines

$\Rightarrow \wedge [\text{int}]$
→ (i|n|t)

$\wedge [\text{int}]^2 \rightarrow$ two locations.

means now if (-val == 0) wont highlight

$\Rightarrow \text{not } i \quad [\wedge i]^2 \rightarrow$ no i on start first two characters

$\Rightarrow \backslash \rightarrow$ check as it is

$\backslash ($ won't work as precedence it will check where is parenthesis.

$\Rightarrow / \rightarrow$ contatenation

x_1 / x_2

$\backslash ?]$ as a ^{exact match} character
? $\rightarrow$ operator.

our Flex $\rightarrow$ keywords | identifiers | numbers |
punctuations | literals | operators

keywords $\rightarrow$ int | float | for | if | printf

letters $\rightarrow [a-zA-Z]$

* hyphen with square brackets.

letter_ $\rightarrow$ letters | _

identifiers $\rightarrow$ letter_ (letter_ | digit)*

punctuations $\rightarrow ; | , | \{ | \} | [|]$

$\Rightarrow$ can write with backslash.

after less than equal
to
?

NO:

DATE:

operator $\rightarrow < | > | < = | > = |$

numbers $\rightarrow [+ -] ? \text{ digit } ^+ (\cdot \text{ digit } ^+) ?$

can be
signed
no or
unsigned
nothing

if
[]
then every
character separate
choice, so

(.) means
anything
(any character)

$[+ -]^0 [+ -]^1$

that's why ()

Transition Diagram

NO: →

FA

DATE: →

→ bring whole code in input buffer

Input Buffer:-

... | - | v | a | l | = | ...

lexeme
begin

↑
forward

digit = {0, 1, 2, ..., 9}

q_0, Q, Σ, δ

letter = {a, b, ..., z, A, ..., Z}

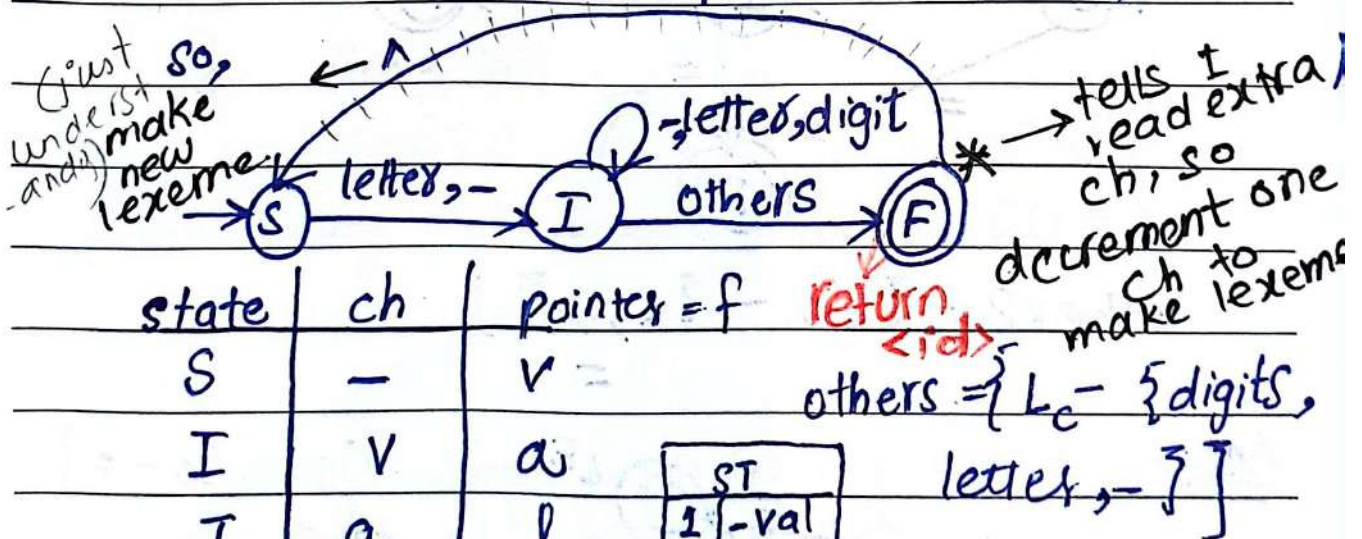
FA → will put ch or set.



→ it will make 1 length lexeme

- v a l in symbol table

final state → lexeme begin pointer moves
to forward place (relocation)

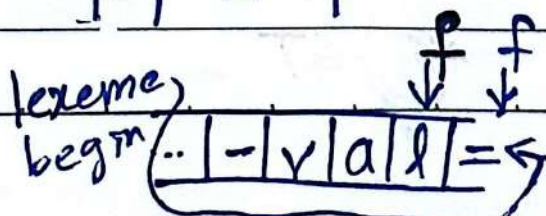


state	ch	pointer = f
S	-	v
I	v	a
I	a	l
I	l	=
F	=	

ST
1 -val

others = {Lc - {digits, letter, -}}

-val = not
our lexeme.



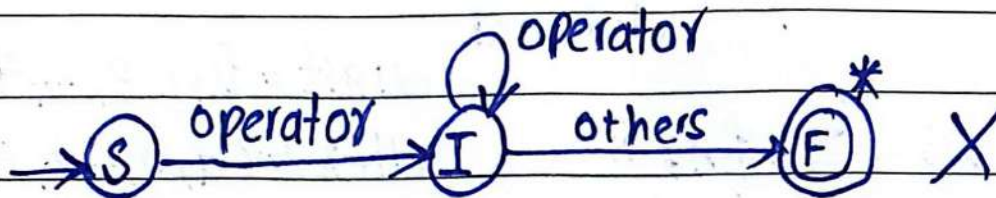
could -val
space

NO: _____

DATE: _____

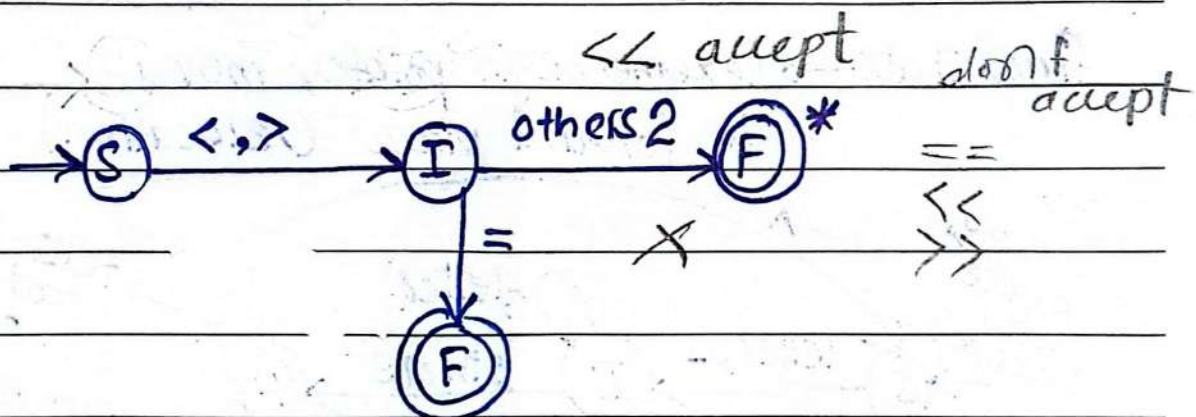
relational operator.

Operators = [$<$, $<=$, $>$, $>=$]



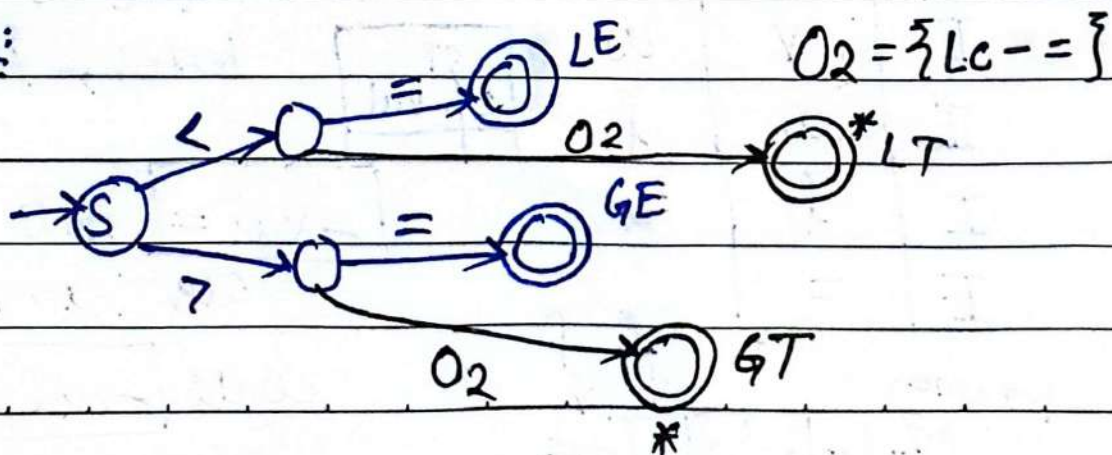
others = $\{Lc - \{<, <=, >, >=\}$

$\Sigma_0 = \{<, >, =\}$ $O_2 = \{Lc - \Sigma_0\}$



others 2 = $\{Lc - \{<, >, =\}$

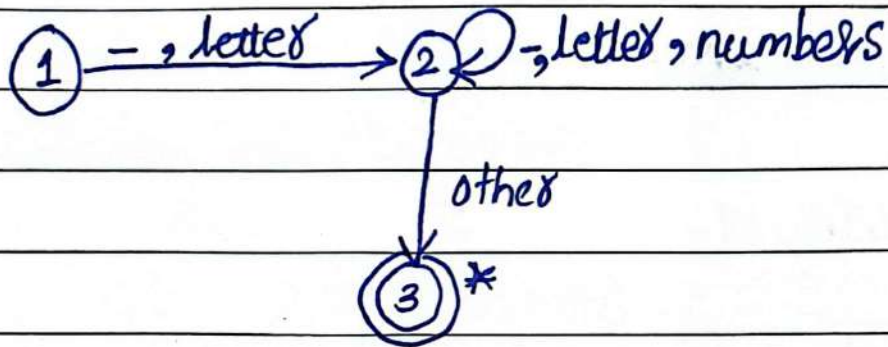
Ans:



FA for identifier

letter = {a, b, c, ..., z, A, B, C, ..., Z}

numbers = {0, 1, 2, ..., 9}



Other = {Σc - {letter, Number, -}}

```
void main() {
```

```
    int state = 1;
```

```
    string IB;
```

```
    cin >> IB;
```

```
    string TokenType = recognizeID(IB);
```

```
    ...
```

```
String recognizeID(string IB) { int state = 1
```

```
    for (char ch : IB)
```

```
        switch (state)
```

```
            case 1:
```

```
                if (ch == '-' || ch >= 'A' && ch <= 'Z'
```

```
                    // ... .. a ... .. z)
```

```
                    state = 2
```

NO: _____

DATE: _____

case 2:

if (ch == '-' || ^{letter} ch ≥ '0' || ch ≤ '9')

state = 2

else

state = 3

default:

state = -1

switch (state)

case 3:

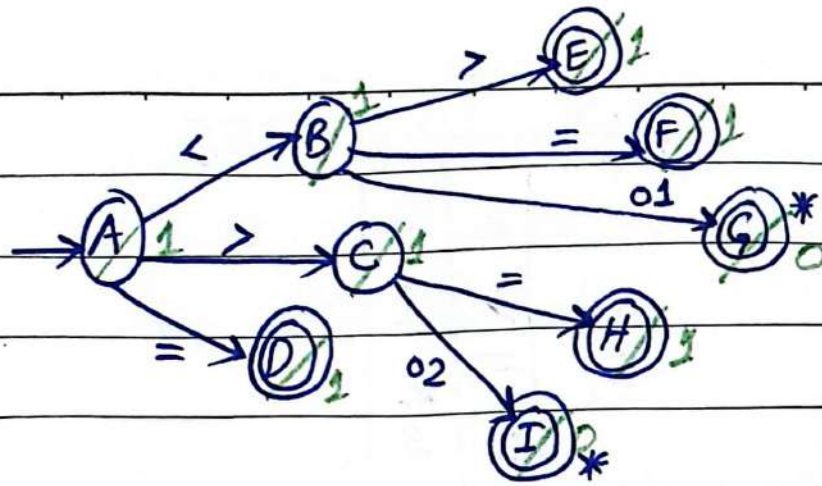
return "<id, string>";

case -1:

return "lexical error";

NO:

DATE:



⇒ (deal *)
advance
move char
forward or not

⇒ < in 01 set
So no need to
write G in
B(<)

Transition Table (T)

 Σ

	</0	>/1	=/2	01/3	02/4	Accept/5
A/0	B/1	C/2	D/3			No/0
B/1	-1	E/4	F/5	G/6		No/0
C/2			H/7		I/8	No/0 sparse
Q D/3						Yes/1 tree.
E/4						Yes/Space
F/5						Yes/compromise
G/6						Yes/1
H/7						Yes/1
I/8						Yes/1

→ table compression techniques.

mapping →
next page

(now 2D matrix)

NO: ∞ Input buffer = 20 ch's.

DATE:

states		Alphabet		FS
A	0	<	0	
B	1	>	1	
C	2	=	2	
D	3	01	3	
E	4	02	4	
F	5			
G	6			
H	7			
I	8			

pg #63
 $\Rightarrow$ algo in book

int state = 0; $\rightarrow$ mapper function

int ch = getnextch (IB); ch map and give index

while (~T[state][5] || ~T[state]) e.g

nextstate = T[state][ch]; ^[error] T[2][2]

if (Advance[state][ch]) Advance[0] return 7 it will

ch = getnextch (IB)

// A state can go to next ch

state = newstate;

if (state == Accept)

return token.

oldstate (12)

advance[1][2]

$\rightarrow$ check if should read nextchar

NO:

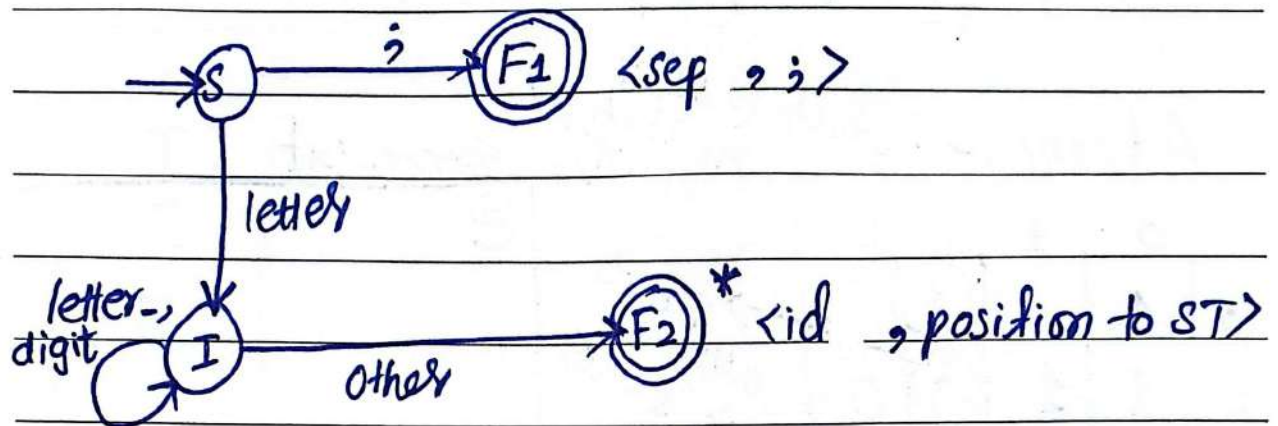
DATE: 20 Feb 25

Transition Table Transform:-

Lex = { identifiers, separator }

identifier = { letter, digit }

separator = { ; }

other = { Σ_c - identifier }

digit = { 0, 1, ..., 9 }

letter = { a, b, c, ..., z, A, B, C, ..., Z }

Step #1:-

TT	letter	digit	;	other
S	I		F1	
I	I	I	F2	F2
F1				
F2				

Step 2: Mapping.

state

0	1	2	3
S	I	F1	F2

Accept (Final)

0	1	2	3
False	False	True	True

Alpha

0	1	2	3
letter	digit	;	other

NO:

DATE:

next state.

	0	1	2	3
0	1		2	
1	1	1	3	3
2				
3				

Token.txt

```
<id,1> <sep,;>
<id,2> <id,3>
<sep,;>
```

Advance → char (to get next ch or not)

	0	1	2	3
0	1		1	
1	1	1	0	0
2				
3				

that's why we * so (0)

Error.txt

@

lexeme = IB[lb: fp]

lexeme begin ← lb

forward pointer ← fp

0: 3

0, 1, 2

Symbol Table.txt

1	num
2	-n
3	um

IB

0	1	2	3	4	5	6	7		
n	u	m	;	-	n	@	u	m	;

lb 1 fp

False: ch: u

accept[1]

State = statemap(s) = 0

ch = getchmap(IB) = 0

while (~Accept[state] & ~error[state])

newstate = T[state][ch];

0 0

1

read next ch

NO: _____

DATE: _____

ch | u | ^m
 if Advance[state][ch] { ch = getnextchmap(18) }
 state = newstate

if Accept(state) { return token; }

state	state-map	ch	ch-map
S	0	n	0
I	1	u	0
I	1	m	0
I	1	;	2

if [1][2] true in Advance table ⁽¹⁾ → keep making identifiers does not separate by ; or operator
 (i) gets in lexeme too.

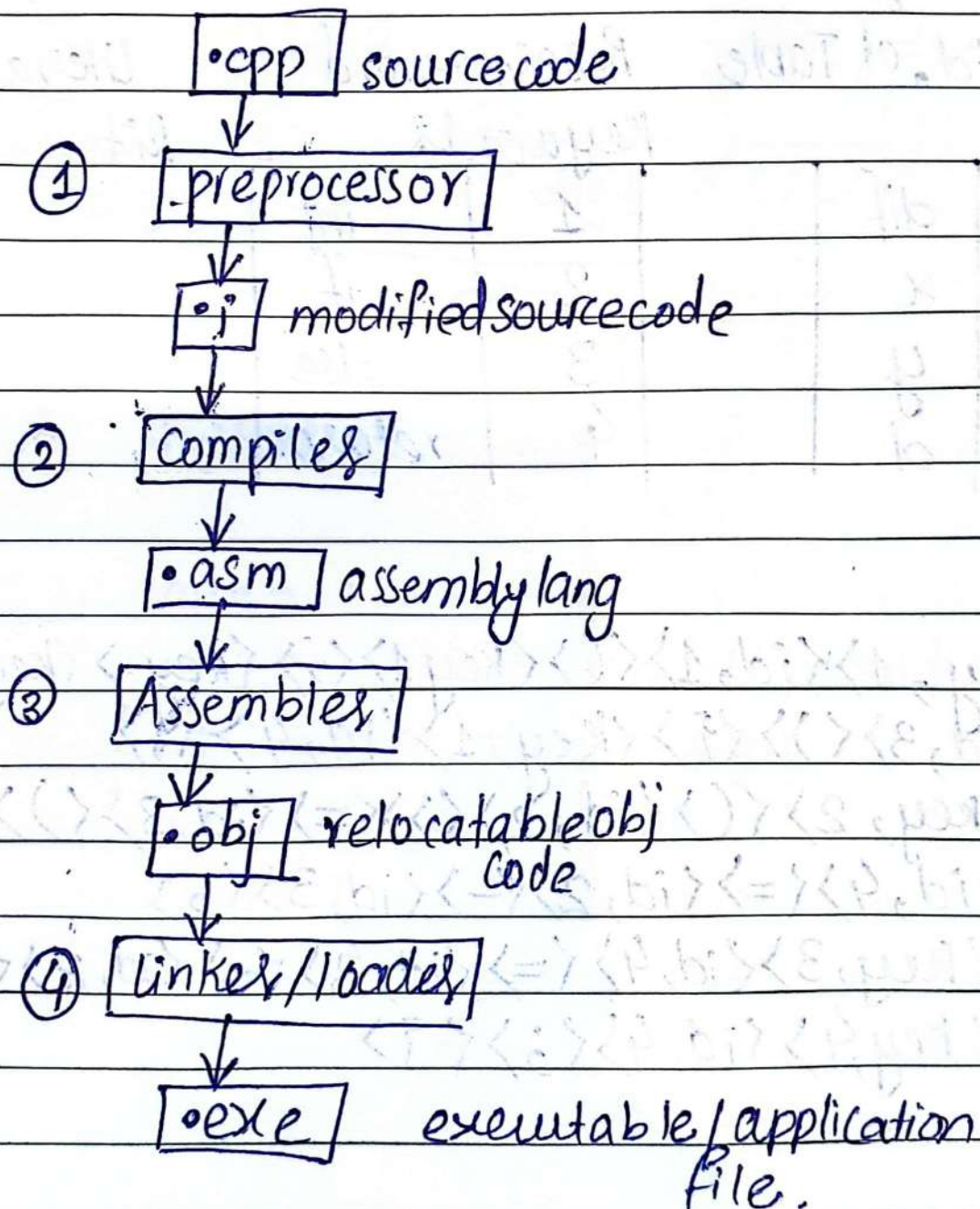
advance
 if [0][2] → (0)
 pointer don't advance,
 infinite (i) in token.txt.

state	state-map	ch	ch-map
S	0	-	0
I	1	n	0
I	1	@	3

in error.txt
 and lb=7 bp=7

Mid# 1 Preparation:- Spring 2024

Q#7: write all phases of compiler



NO:

DATE:

Fall 2023

Q#1: token-lexeme pairs

Symbol Table		Reserve-word		Literal
id		Keywords		lit
1	dif	1	int	
2	x	2	if	
3	y	3	else	
4	d	4	return.	

<key, 1> <id, 1> <()> <key, 1> <id, 2> <?> <key, 1>
 <id, 3> <)> <{> <key, 1> <id, 4> < ;>
 <key, 2> <()> <id, 2> <)> <=> <id, 3> <()>
 <id, 4> <=> <id, 2> <-> <id, 3> < ;>
 <key, 3> <id, 4> <=> <id, 3> <-> <id, 2> < ;>
 <key, 4> <id, 4> < ;> < }>

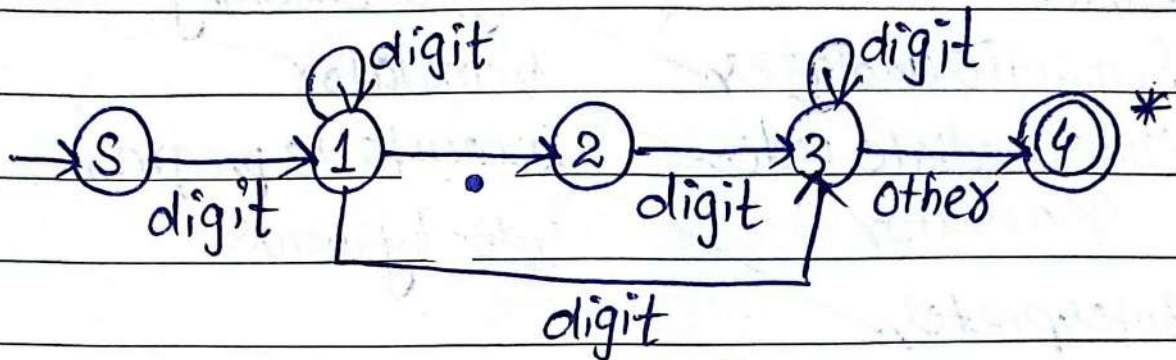
NO:

DATE:

Q#2: DFA for token num

$num \rightarrow digit^+ F$

$F \rightarrow 1 | \cdot digit^+ | n$



other = $\{ \Sigma_c - \{digit\} \}$

digit = $\{ 0, 1, 2, \dots, 9 \}$

Q#3: parse tree

$S \rightarrow S \& S \mid S = S \mid S \# S \mid id \mid num \mid$

$\& \#$ higher than $=$ sign.

NO:

DATE: Fall 2021:

Q#1: a)

- | | |
|--------------------------------|-----------------------------|
| a) Lexical analyzer | Convert word into sentences |
| b) Parser | Type checking |
| c) Semantic analyzer | translator |
| d) Intermediate code generator | execute program |
| e) Interpreter | identify words. |

b)

compiler provides fast execution

Interpreter does not hide source code

hybrid provide platform independence

interpreter is not good for debugging

hybrid uses both compiler & debugger

Q#2: regular definition:- (a)

09:10:40 am 11:05:09 pm

$H(1) \rightarrow 0|1$

$M(1) \rightarrow 0|1|2|3|4|5$

$H(2) \rightarrow [0-9]$

$M(2) \rightarrow [0-9]$

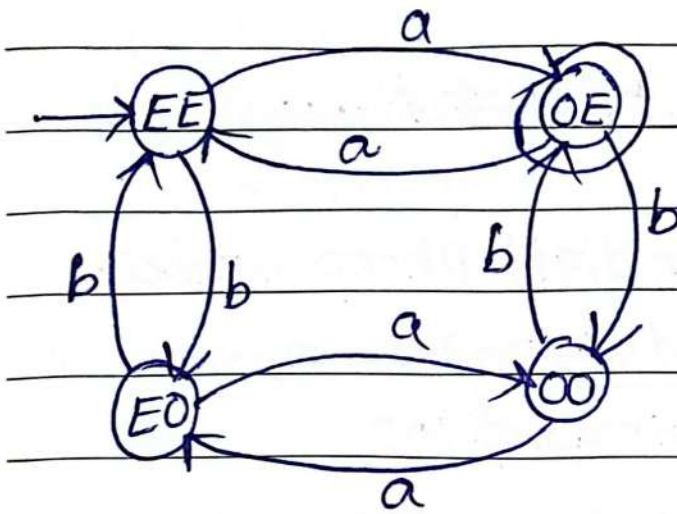
$F \rightarrow am/pm$

$H \rightarrow H(1) H(2)$

$M \rightarrow M(1) M(2)$

Time $\rightarrow H:M:MF$

(b) odd # of a's
even # of b's



Fall 2020: (B) MCQs

(i) compiler $\rightarrow$ fast execution and source code protection

(ii) interpreter $\rightarrow$ better debugging, portability

(iii) hybrid $\rightarrow$ All of above

(iv) populate symbol table $\rightarrow$ lexical analyzer, semantic analyzer.

(v) RE cannot include $\rightarrow$ recursion.

NO:

DATE:

a) regular definition

* 234 * 6214532 * 500 # *

234 fixed seven digit phone number
between 100 and 1000
multiples of 100

code $\rightarrow$ 234

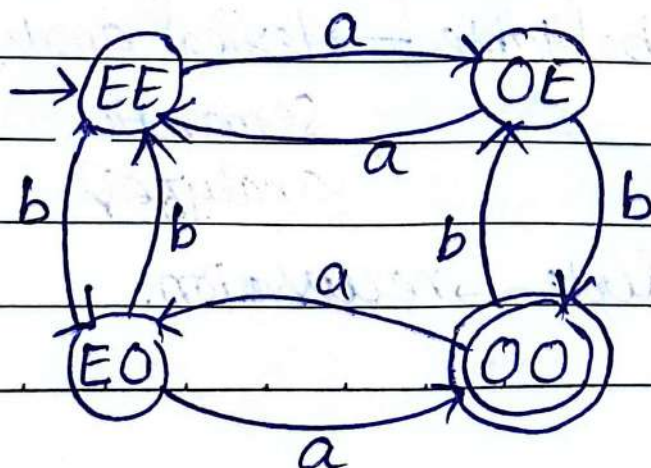
digit $\rightarrow$ [0-9]

no $\rightarrow$ digit . digit . digit . digit . digit . digit . digit

amount $\rightarrow$ 200 / 300 / 400 / 500 / 600 / 700 / 100 /
800 / 900 / 1000

service code $\rightarrow$ * code * no * amount #

b) DFA for odd a and odd b



Spring 2020:

Q#1(a):

- 1) Regular definition cannot contain recursion
- 2) code protection → compiler and hybrid
- 3) Symbol table → identifiers
- 4) portability (platform independence)
↓
interpreter and hybrid.
- 5) hybrid → compiler and interpreter

b) Match entries.

- | | |
|-------------------------|--|
| a) optimization | (i) converts word to sentence |
| b) Parser | (ii) type checking |
| c) Semantic analyzer | (iii) partial evaluation of boolean expression |
| d) short-cut evaluation | (iv) execute program |
| e) interpreter | (v) use less time and space |

NO: _____

DATE: _____

Q#2: Regular definition:
for

29-Feb-2020

31-Mar-1982

15-Dec-2010

Month $\rightarrow$ Jan | Mar | Apr | May | June | July | Aug |
Sep | Oct | Nov |

Date $\rightarrow$ [01-31] digit $\rightarrow$ [0-9]

Year $\rightarrow$ digit digit digit digit

Date $\rightarrow$ date - Month - Year /
(a) [01-29] - Feb - Year

Fall 2019:

v) Two diff parse trees for
Single input $\rightarrow$ ambiguous grammar

(vi) lexical analyzer $\rightarrow$ converts
lexemes $\rightarrow$ tokens

NO:

DATE:

(vii) syntax analyzer
converts

words $\rightarrow$ sentences.

(ix) syntax error $\rightarrow$ syntax analyzer

Fall 2018:

LA $\rightarrow$ type? False

parser $\rightarrow$ fun (# of parameters)? False

[Short-circuit] $\rightarrow$ error False

evaluation

$\downarrow$ optimization technique.

intermediate code $\rightarrow$ SAC True.

Semantic analyzer $\rightarrow$ type
check True.

NO:

DATE:

Regular Definition:-

12 Feb 2025

string variable = "Hi"

all three (LA, SA, SemA)
errors

variable

Q) variables can start with vowels

vowels $\rightarrow$ a|e|i|o|u
 $\rightarrow$ identifiers $\rightarrow$ vowels] x

identifier $\rightarrow$ [a|e|i|o|u] (letter|digit)*
 $\downarrow$ choice added.
(a|e|i|o|u)

Q) numbers' regular definition

$\rightarrow$ [+ -]? [0, 1]

number $\rightarrow$ [+ -]? digit +

number $\rightarrow$ [+ -] [0, 1] digit +
[range]

NO:

DATE:

as identifier starts with vowels

so ab $\rightarrow$ id $\checkmark$

course X invalid $\rightarrow$ lexical error

2ab $\rightarrow$ NOT a lexical error
 $\downarrow$ digit $\downarrow$ id starts with vowel a

Special number:-

start with @ or # from pos 2 to 5
can have numbers (1, 3, 7) and after
that any no can come.

e.g

@13, @37, @7300, @11100
@137, @1377, #1377701

OR

min 2, 3 pos per [lazmi] abc mein se
kuch asakta ha @1ab X

and max 2-5 tak abc a sakta ha
is ke baad age digits aye gay.

Sp number $\rightarrow$ [$\#$ @][abc]²([abc]³)digit*

OR

for two location $\downarrow$ location
1, 2 $\downarrow$ 3, 4, 5

NO:
DATE:

"." for string
literal

Spnumber $\rightarrow$ $[\# @] [abc] [2, 5]$ digit $*$

String with anything in it

String $\rightarrow$ $[" . * "]$

$\bullet \rightarrow$ means
everything

Q) select all words who
do not have 0-9

nodigit $\rightarrow$ $[^0-9] +$
 $\hookrightarrow$ negation ①

if: $^ [0-9] +$ if line starts
with 0-9 select line ②

$(^ int) \rightarrow$ select all lines that
starts with int

$^ [int] \rightarrow$ select lines that
starts with i, n, t

NO:

DATE:

*
end → \$

;\$ (select all lines that
ends with ;)

backwardslash - \

forward slash /

NO:

DATE:

4th March 2025

(Lexical Phase)

Phase #1

10 March

Monday

CFG
Syntax Analysis

```
void main() {
```

```
    int a = 10;
```

```
    int b = 10;
```

```
    int c;
```

```
    c = a * b + b;
```

token

getNextToken()

Lexical

RE/RD/FA's

CFG: 4 tuples

Σ, V, S_r, P

terminal, variable, start variable, production

Token

$P \rightarrow \alpha$ CFG $V \in P \rightarrow \text{derivable}$

$P ::= \alpha$ BNF

RD/CFG

(VUS)

$d_i \rightarrow s_i$

regular definition

regular exp

$a^n b^n \quad n \geq 0$

$S \rightarrow aSb$

(1)

because of this *

recursive relation

base case

$\Rightarrow s_i$ doesn't contain d_i

Non-Left Recursive Grammar

```
S(n) { if(n == 0) return;
```

```
    else
```

```
        cout << "a";
```

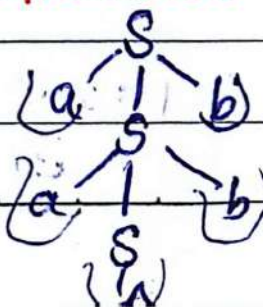
```
        S(n-1);
```

```
        cout << "b";
```

```
}
```

CFG₁

CFG₂



$(ab)^n$

NO:

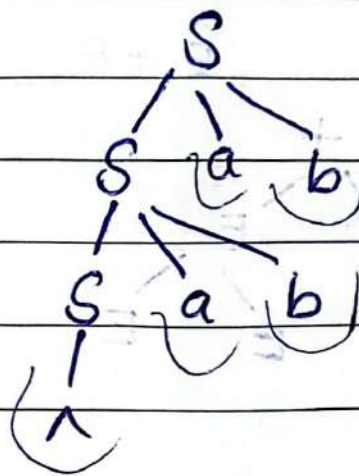
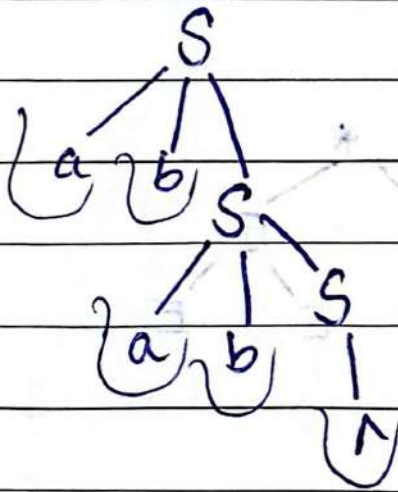
DATE:

CFG #1

CFG #2

$S \rightarrow abs / \Lambda$

$S \rightarrow sab / \Lambda$



= ababΛ

= abab

= Λabab

= abab

① Right Recursive

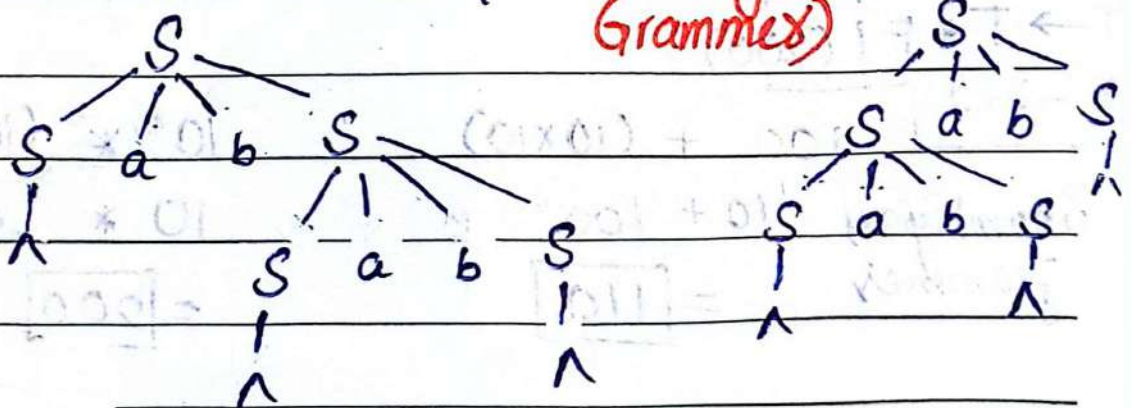
↓ grows on right side

② Left Recursive

↓ grows on left side

$S \rightarrow SabS / \Lambda$

③ (Ambiguous Grammars)
↑ harmful.



*We will write grammar that should be unambiguous.

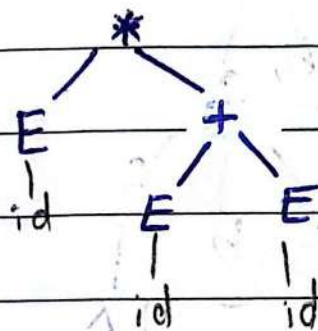
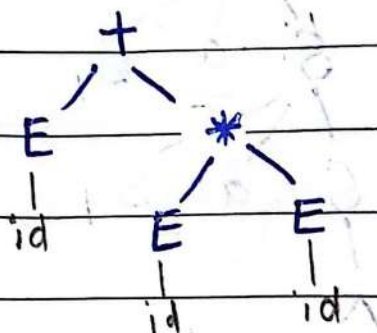
NO: $\Rightarrow$ ambiguous grammar

DATE: $\downarrow$

$E \rightarrow E + E \mid E * E \mid \langle id \rangle$

1 2 3

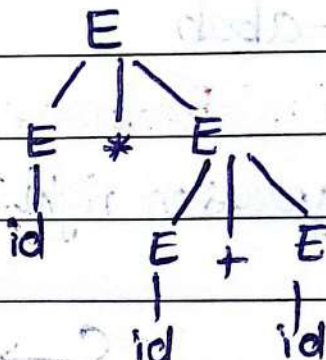
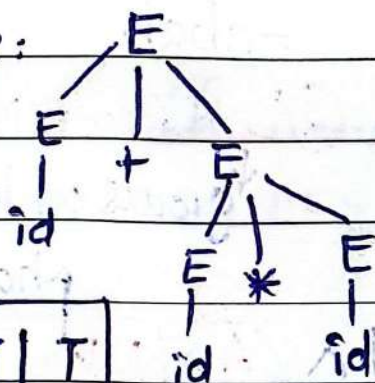
or id



Correct:-

precedence:

$*, +$
(left
associative)
op



$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

$10 + (10 \times 10)$

$10 * (10 + 10)$

$10 + 100$

$10 * 20$

unambiguous
grammars

$= 110$

$= 200$

$\Rightarrow$ substitution key like we get

only one path.

$S \rightarrow Sab$ or e.g

i.e unambiguous grammar

$S \rightarrow abS$

NO:

DATE:

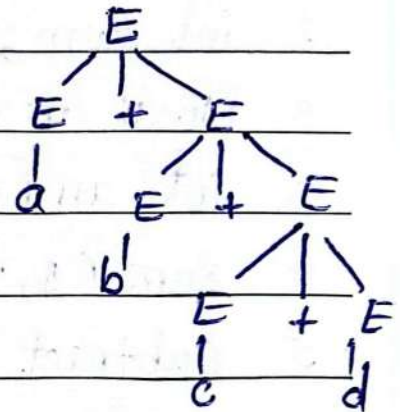
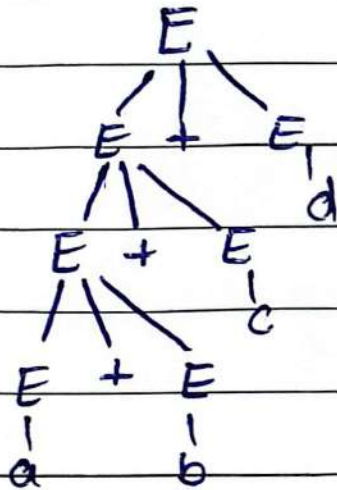
$((a+b) + c) + d$

if left Recursive

$(a+(b+(c+d)))$

if Right Recursive

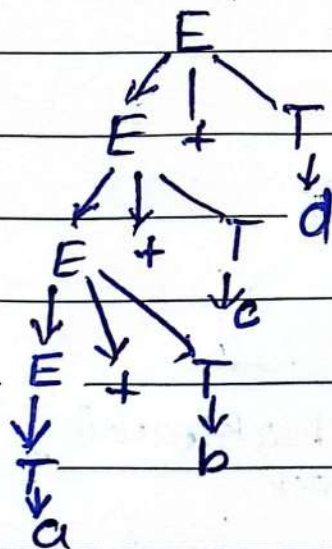
Associativity



$\Rightarrow E \rightarrow E + T \mid T$
 $T \rightarrow id$

non-ambiguous
grammars.

(left recursive)
grammars



$\Rightarrow$ right associative

$E \rightarrow T + E \mid T$
 $T \rightarrow id.$

Left Factoring:

NO:

DATE:

6 March 2025

remove backtracking
from grammar

CFG

declaration:-

• cpp

1 int num;

<id>

2 float array [10];

<num>

3 int arr [1,2]; // 2D array

4 sum (1,2);

5 subtract (1,2,3);

keyword $\rightarrow$ int | float

identifier $\rightarrow$ a | b | c | ... | z | A | B | C | ... | Z

num $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | ,

id $\rightarrow$ a | b | c | ... | z | A | B | C | ... | Z

K $\rightarrow$ int | float

no
need
to
write

✓ code $\rightarrow$ K id ; | K id [num] ; | id (num) ;

OR: multiple ways $\rightarrow$ backtracking (tree fails go back)

✓ Line $\rightarrow$ ^{for line 1} keywords ^{& backtrack} id [num param] ; | keyword id ;

id (fun Parameters) ; * efficiency error.

Keyword $\rightarrow$ int | float

num param $\rightarrow$ num, num param

id $\rightarrow$ <id>

$\Rightarrow$ multiple trees, fail etc. | num.

fun Parameters $\rightarrow$ id, fun Parameters | num, fun Parameters

num $\rightarrow$ <num>

id | num.

not part of tree
NO: of derivation

DATE:

OR: $\rightarrow$ no backtracking

correct

$S \rightarrow K \langle id \rangle B ; \mid \langle id \rangle (P) ;$
 $K \rightarrow \text{int} \mid \text{float}$
 $B \rightarrow [P] \mid \Lambda$
 $P \rightarrow \langle \text{num} \rangle \mid \langle \text{num} \rangle , P$

(lookahead of one word)

condition checking

num if $\checkmark$ do else
no

$\rightarrow$ for line (3) int arr [1, 2];

OR: $S \rightarrow A \mid B \mid F$

fail then go
to $\langle \text{num} \rangle , P$

$K \rightarrow \langle \text{int} \rangle \mid \langle \text{float} \rangle$

$A \rightarrow K \langle id \rangle \langle ; \rangle$

$\Rightarrow$ indirect backtracking $B \rightarrow K \langle id \rangle \langle [\rangle D \langle] \rangle \langle ; \rangle$

$E \rightarrow \langle \text{sum} \rangle \mid \langle \text{subtract} \rangle$

$F \rightarrow E \langle (\rangle D \langle) \rangle \langle ; \rangle$

$D \rightarrow \langle \text{num} \rangle \langle , \rangle D \mid \langle \text{num} \rangle$

Left Factoring:-

$\alpha A = \alpha B$ but not always

$\alpha A \rightarrow \alpha_B B_1 \mid \alpha_B B_2 \mid \alpha_B B_3 \dots$
 $\gamma_1 \mid \gamma_2 \mid \gamma_3 \dots$

$\Rightarrow \alpha A \rightarrow \alpha_B B / \gamma$

$B \rightarrow B_1 \mid B_2 \mid B_3$

$P \rightarrow \langle \text{num} \rangle \mid \langle \text{num} \rangle P$
 $\alpha_A \quad \alpha_B \quad \beta_1 \quad \alpha_B \quad \beta_2$
 $\downarrow$
 Λ

$P \rightarrow \langle \text{num} \rangle P'$

$P' \rightarrow \Lambda \mid , P$

} new grammar

NO:

DATE:

↑ longest matching prefix *

$\alpha\beta$ β_1 $\alpha\beta$ β_2

line $\rightarrow$ keyword id [num Param]; | keywords id ; |
 αA id (fun Parameters);
 γ_1

line $\rightarrow$ keyword id (line); | id (funParameters);

(line)' $\rightarrow$ [num Param]; . ;

$\alpha A \rightarrow \alpha\beta\beta_1 / \gamma_1$

$\beta \rightarrow \beta_1 / \beta_2$

$\Rightarrow$ prefix shouldn't be matched for any grammar

e.g. $E \rightarrow E * T \mid E + T \mid T$ multiple productions
 $T \rightarrow id \mid num$ prefix is same.

NO:

DATE:

11 March 2025

LL(1) Grammar:

- ↓ Left Factored (i)
- ↓ Non-ambiguous (ii)
- Single path (iii)
- Non-left Recursive

Solved by

Left Factoring

Grammar issue

backtracking ①

efficiency destroyed.

② Ambiguity problem

uncertain results.

(right recursive or left)

Right side of grammar

Variable is called more than once

e.g. $S \rightarrow SabS \mid \Lambda$

More than one tree for single line

③ Left Recursion

when self call is on left most side

e.g. $E \rightarrow E + T \mid T$
 $T \rightarrow id \mid num$

$S \rightarrow Sab \mid \Lambda$

$S() \{$

first line

$\leftarrow S() ;$

$\rightarrow$ stack full $\rightarrow$ stack

overflow

infinite loop;

$S \rightarrow abS \mid \Lambda$

① a | b | a | b

② $S(call)$

$S() \{$

check a
check b

$S()$

not stuck now

}

NO:

DATE:

$$E \rightarrow E + T \mid T$$



if changed to

$$E \rightarrow \boxed{T + E} \mid T$$

*bonus.

issue

language
change.

$$A \rightarrow A \alpha \mid B$$

B

B α

B $\alpha\alpha$

$\boxed{B\alpha^*}$

$$A \rightarrow \alpha A \mid B$$

B

αB

$\alpha\alpha B$

$\boxed{\alpha^* B}$

① Invalid words in language.

②

$$\underline{a+b}+c+d$$

$$a+b+\underline{c+d}$$

operator associativity

issue

e.g

$$2^{3^4}$$

actually
right

$$2^{3^4}$$

ans

→ in actual

$$2^{(3^4)}$$

issue if switch

$$2^{3^4}$$

ans

NO:

DATE:

$$A \rightarrow A\alpha \mid B$$

Non Left Recursion.

$$\begin{array}{l|l} A \rightarrow BA' & (1) \\ A' \rightarrow \alpha A' \mid \Lambda & (2) \quad (3) \end{array}$$

$$B \quad (1)$$

$$B\alpha \quad (123)$$

$$B\alpha\alpha \quad (1223)$$

$$\alpha B \quad (213)$$

$$\begin{array}{l} \textcircled{1} \quad \begin{array}{l} \overbrace{E \rightarrow E + T}^A \mid \overbrace{T}^B \\ T \rightarrow id \mid num \end{array} \Rightarrow \begin{array}{l} E \rightarrow TE' \\ E' \rightarrow +TE' \mid \Lambda \end{array} \end{array}$$

transforms.

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \Lambda$$

$$T \rightarrow id \mid num$$

$$\textcircled{2} \quad \begin{array}{l} \overbrace{S \rightarrow S a b}^A \mid \overbrace{\Lambda}^B \end{array}$$

$$S \rightarrow S'$$

$$S' \rightarrow abS \mid \Lambda$$

NO: _____
DATE: _____

ch # 4, 5

$$\begin{array}{l} A \rightarrow A\alpha_1 \mid A\alpha_2 \mid B_1 \mid B_2 \\ A \rightarrow \cdot B_1 A' \mid B_2 A' \\ A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \Lambda \end{array}$$

13 March 2025

→ Check if LL(1) grammar

$$\begin{array}{l} S \rightarrow Sab \mid Ab \\ A \rightarrow aS \mid a \end{array}$$

↓ useful in
Top down
Parser.

- Ambiguity
- Back Tracking (Left Factoring) → 3, 4
- Left Recursion (Elimination) → 1

$$\begin{array}{l} \overline{S} \rightarrow \overline{S}ab \mid \overline{A}b \\ \overline{A} \rightarrow \overline{a}S \mid \overline{a} \end{array}$$

Left Recursion:-

$$\begin{array}{l} A \rightarrow A\alpha \mid B \\ A \rightarrow BA' \\ A' \rightarrow \alpha A' \mid \Lambda \end{array}$$

Ans

$$\begin{array}{l} \Rightarrow \overline{S} \rightarrow Ab\overline{S}' \\ \overline{S}' \rightarrow ab\overline{S}' \mid \Lambda \end{array}$$

Left Factoring:-

$$\begin{array}{l} \alpha \rightarrow \alpha B_1 \mid \alpha B_2 \mid \gamma_1 \mid \gamma_2 \\ \alpha \rightarrow \alpha B \mid \gamma_1 \mid \gamma_2 \\ B \rightarrow B_1 \mid B_2 \end{array}$$

$$A \rightarrow aB$$

$$B \rightarrow S \mid \Lambda$$

forward substitution

⇒ we need NO:

cycle free grammar

$S \rightarrow a b S \mid A b$

$A \rightarrow S \rightarrow A$

(cycle)

$A \rightarrow S a \mid a$

↓

$A \rightarrow a S \mid a$
(not cycle)

$S() \rightarrow$

$S() \rightarrow$ infinite loop.

decision making choice.

Left Recursion of
(2) types

$S() \rightarrow$

$A() \rightarrow$

$A()$

$S()$

cycle

infinite loop and Stack overflow.

Direct
↓
Self call

Indirect
↓
cycle.

causes ambiguity tho.

backward substitution

Substitution method

→ one option

Substitute A

$S \rightarrow a b S \mid S a \mid a$

(preferable)

→ or substitution S

Forward substitution

$A \rightarrow a b S a \mid A b a \mid a$

indirect converted to direct

NO:

DATE:

after left recursion elimination

$$\begin{array}{l}
 \beta_1 \quad A \quad \alpha \quad \beta_2 \\
 A \rightarrow abSa \mid A ba \mid a \\
 S \rightarrow abS \mid Ab \\
 A \rightarrow aA' \mid abSaA' \\
 A' \rightarrow baA' \mid \Lambda
 \end{array}$$

$$\begin{array}{l}
 S \rightarrow abS \mid Ab \\
 A \rightarrow aA'' \\
 A'' \rightarrow A' \mid bSaA' \\
 A' \rightarrow baA' \mid \Lambda
 \end{array}$$

after left factoring.

Ans

Left Factoring

Backward Substitution

Direct

Indirect

Common Prefix

Multiple Levels of grammar.

if $A'' \rightarrow A' \mid bSaA'$
 $A' \rightarrow b'aA' \mid \Lambda$
 $B' \rightarrow baA'$
Yes

if $A'' \rightarrow A' \mid (bSaA'$
 $A' \rightarrow (baA' \mid \Lambda$ Yes
 if $A'' \rightarrow A' \mid (bSaA'$ No
 $A' \rightarrow (baA' \mid \Lambda$

after
backward substitution

NO:

DATE:

$$A'' \rightarrow b a A' \mid \Lambda \mid b S a A'$$

$$A' \rightarrow b a A' \mid \Lambda$$

$$S \rightarrow \overset{\beta_1}{a} \overset{\beta_2}{b} S \mid \overset{\beta_1}{a} \overset{\beta_2}{A''} b$$

$$A \rightarrow a A''$$

need?

Discard $\rightarrow$ **NOT**

reachable from

Start

$$S \xrightarrow{1} a S'_2$$

$$S'_2 \xrightarrow{3} b S'_3 \mid A'' b$$

$$A'' \xrightarrow{4} b A''' \mid \Lambda$$

$$A''' \xrightarrow{6} a A'_7 \mid S a A'_7$$

$$A'_7 \xrightarrow{8} b a A'_8 \mid \Lambda$$

(3,4) cycle



Recursive Descent Parser :-

NO:

DATE:

20 March 2025

left associative
↓

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid$

Self call left side

$ID \mid NUM \mid (E)$

③
ambiguity removed:-

$E \rightarrow E + T \mid E - T \mid T$

$T = Texm$

$F = factor$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow ID \mid NUM \mid (E)$

$a * 10 + (b + c)$

① left factoring:-

$\alpha A \rightarrow \alpha B B_1 \mid \gamma_1 \mid \alpha B B_2$

$\alpha A \rightarrow \alpha B B \mid \gamma_1$

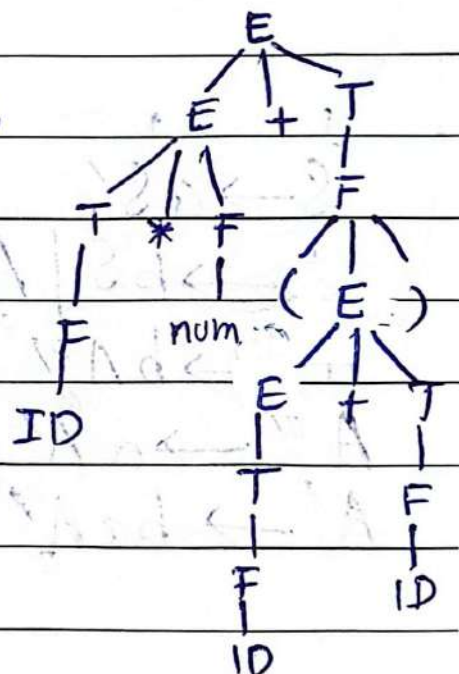
$B \rightarrow B_1 \mid B_2$

② left recursion:-

$A \rightarrow A \alpha \mid B$

$A \rightarrow B A'$

$A' \rightarrow \alpha A' \mid \Lambda$



after ②

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid - T E' \mid \Lambda$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid / F T' \mid \Lambda$

$F \rightarrow ID \mid NUM \mid (E)$

NO:

DATE:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid -TE'$$

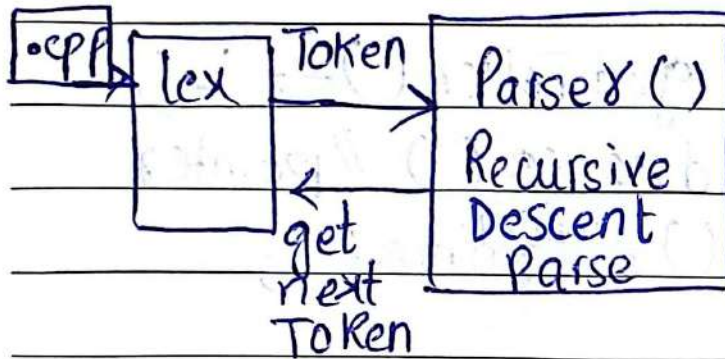
$$E' \rightarrow \wedge$$

$$T \rightarrow ID \mid NUM$$

→ put in loop → can or stream of tokens.

matching condition

↑
terminals name



function calls.
↓
variables

10 + NUM

ParserX() { E(); } (write rule left to right)

E() { T(); Eprime(); }

T() { if (get token() == ID)

Advance();

call to lexical analysis. else if (get token() == NUM) Advance();

else

"error"; → parsing error or syntax error.

void main() { ParserX(); }

cout << "Successful Parsing";

NO:

DATE:

→ condition check and fail not backtracking

```
Eprime() {  
    if (gettoken() == '+')  
    {  
        Advance();  
        T(); Eprime();  
    }
```

```
    else if (gettoken() == '-')  
    {  
        Advance(); // pointer  
        T(); Eprime();  
    }
```

```
    else return; // ^  
}
```

10	+	NUM	eof
----	---	-----	-----



NO:

DATE:

stmt $\rightarrow$ if Exp then stmt |
if Exp then stmt else stmt |
other

$\rightarrow$ dangling else problem.

if E1 then if E2 then S1 else S2

Parse tree :- 1, 2

or

dangling else problem.

2 1

$\downarrow$ can have
problem
with if else.

$\rightarrow$ pseudocode

$\hookrightarrow$ else is attached
with nearest if.

NO:

DATE:

Attributed grammar

= (CFG +

24 March 2025

Semantic Rules

↓ we need
grammar.

$A = 3 * 4 + 5;$

to differentiate parent and child E
 $E①$

Production:

Semantic Rule:

① $L \rightarrow E_n$ $L.val = E.val$ (action)

② $E \rightarrow E_1 + T$ $E.val = E_1.val + T.val$

③ $E \rightarrow T$ $E.val = T.val$

④ $T \rightarrow T_1 * F$ $T.val = \text{lower level } T_1.val \times F.val$

⑤ $T \rightarrow F$ $T.val = F.val$

top to bottom

Two terms $\rightarrow$ expansion &

reduction

⑥ $F \rightarrow (E)$ bottom up. $F.val = E.val$

⑦ $F \rightarrow \text{digit}$ $F.val = \text{digit}.lexval.$

S-attribute grammar

→ E.val, F.val, T.val
NO:
DATE:

Synthesize attribute :-

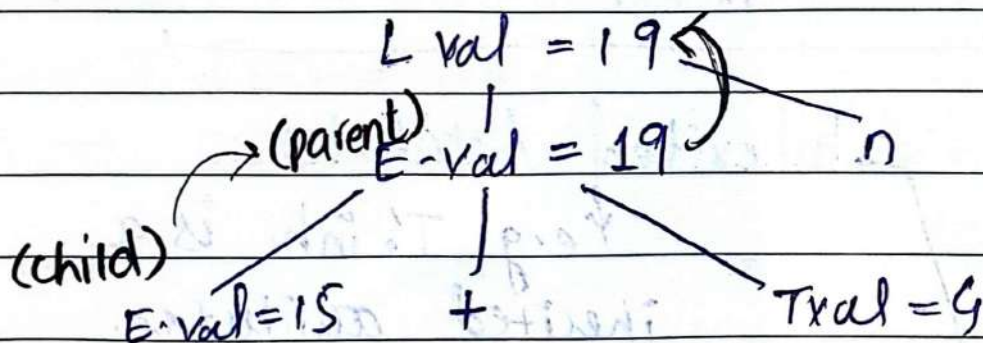
↓
value comes from child.

every attribute dependendent on
child to compute value

S-attribute grammar

→ all attributes are synthesize

Syntax directed Definition :-

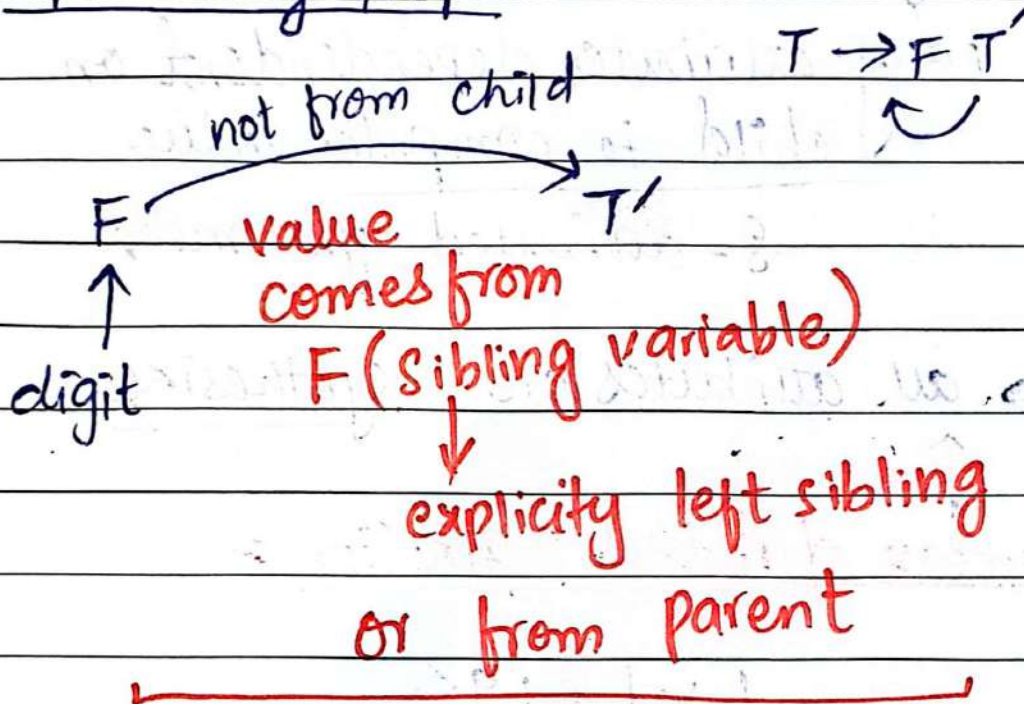


NO:
DATE:

S-Attributed (Synthesise) + Inherited Attributed
rule fire reduction (bottom up)

L-Attributed Grammar:-

Dependency Graph:-



Inherited Attribute

e.g. $T.inh$ is a inherited attribute

top down (expand production)
rule fire

Syntax directed definition $\rightarrow$ exact rule not know
figuring out based on
attribute type

SDD

① $T \rightarrow F \{0\} T' \{2\}$
 hone se pehle
 ye hoga

NO:

DATE:

- ① $T' \text{ inh} = F \cdot \text{val}$
 ② $T \cdot \text{val} = T' \cdot \text{syn}$ } two rules how
 will it
 compute values?

SDT → rules given at exact location
 → production or rule is given sath sath
 syntax directed translation explicitly rule present (localized)

SDT:

26 March 2025

$$3 + 4 * 5 = 23$$

$L \rightarrow E ; \{ L.v ; \text{print}(L.v) \}$

$E \rightarrow E1 + T \{ E.v = E1.v + T.v \}$

$E \rightarrow T \{ E.v = T.v \}$

$T \rightarrow T1 * F \{ T.v = T1.v * F.v \}$

$$3 * 5 + 4 \text{ (n)}$$

terminator

NO:
DATE:

⇒ Annotated parse tree → attributed values present.

$$1 \quad T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$$

$$2 \quad T' \rightarrow * F \{ T'.inh = T'.inh * F.val \}$$

$$T'_1 \{ T'.syn = T'.syn \}$$

$$3 \quad T' \rightarrow null \{ T'.syn = T'.inh \}$$

$$4 \quad F \rightarrow digit \{ F.val = digit.lexval \}$$

mid # → Suppose to write (SDTS):
write localized rule

Q# we have to count # of 1's
in a binary string. output
e.g. 0101011 ⇒ 4

CFG?

$S \rightarrow OS \mid 1S \mid \wedge$ (string generate)

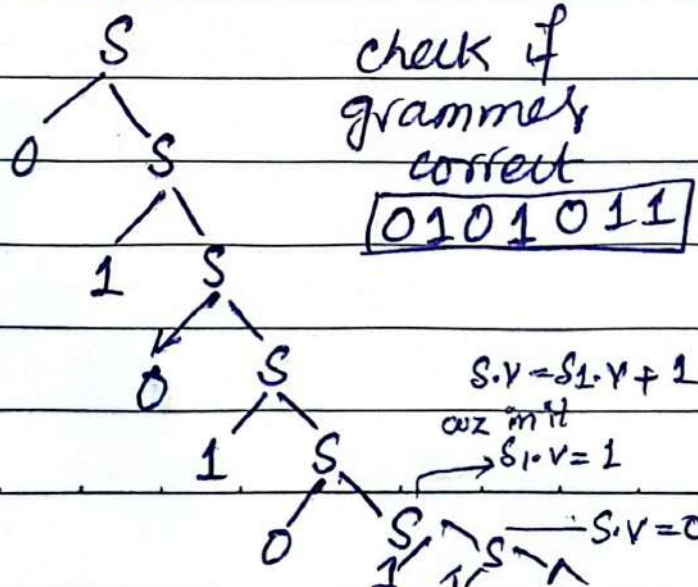
① CFG

$$S \rightarrow OS$$

$$S \rightarrow 1S$$

$$S \rightarrow \wedge$$

② Parse Tree



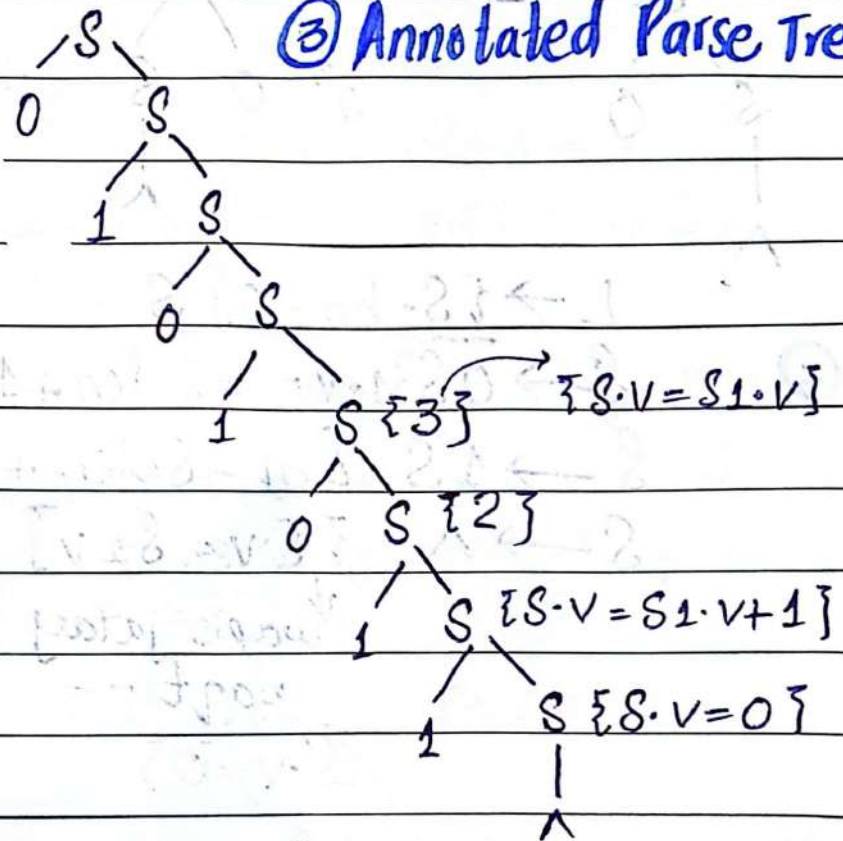
DATE:

to know its child.

$$S \rightarrow 1S_1 [S.v = S_1.v + 1] \rightarrow \text{count \# of}$$
$$S \rightarrow \wedge \{S.v = 0\}$$

1's.

③ Annotated Parse Tree



Q# Now we have to write an SDT that will compute decimal equivalent number for binary numbers.

① CFG

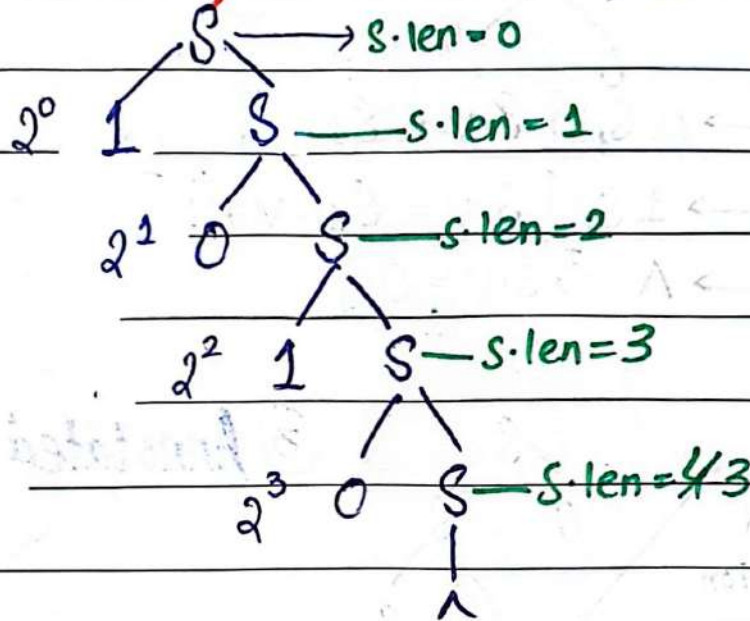
$$S \rightarrow OS_1$$
$$S \rightarrow 1S_1$$
$$S \rightarrow \wedge$$
$$(0101)_2 = (5)_{10}$$

NO:
DATE:

not a
U1
↓
grammar

$$(0101)_2 = (5)_{10}$$

②
Parse
tree



$$L \rightarrow \{S \cdot \text{len} = 0\} S$$

③

$$S \rightarrow 0 \{S_1 \cdot \text{len} = S \cdot \text{len} + 1\} S_1 \{S \cdot v = S_1 \cdot v\}$$

$$S \rightarrow 1 \{S_1 \cdot \text{len} = S \cdot \text{len} + 1\} S_1$$

$$S \rightarrow \wedge \{S \cdot \text{len} --\}$$

↓
wapis jatai
wagt --

$$\{S \cdot v = 0\}$$

$$\{S \cdot v = S_1 \cdot v + 2^{S \cdot \text{len}}\}$$

$$L \rightarrow \{S \cdot \text{len} = 0\} S$$

$$S \rightarrow 0 \{S_1 \cdot \text{len} = S \cdot \text{len} + 1\} S_1$$

$$S \rightarrow 1 \{S_1 \cdot \text{len} = S \cdot \text{len} + 1\} S_1 \{S \cdot \text{len}$$

Answer:-

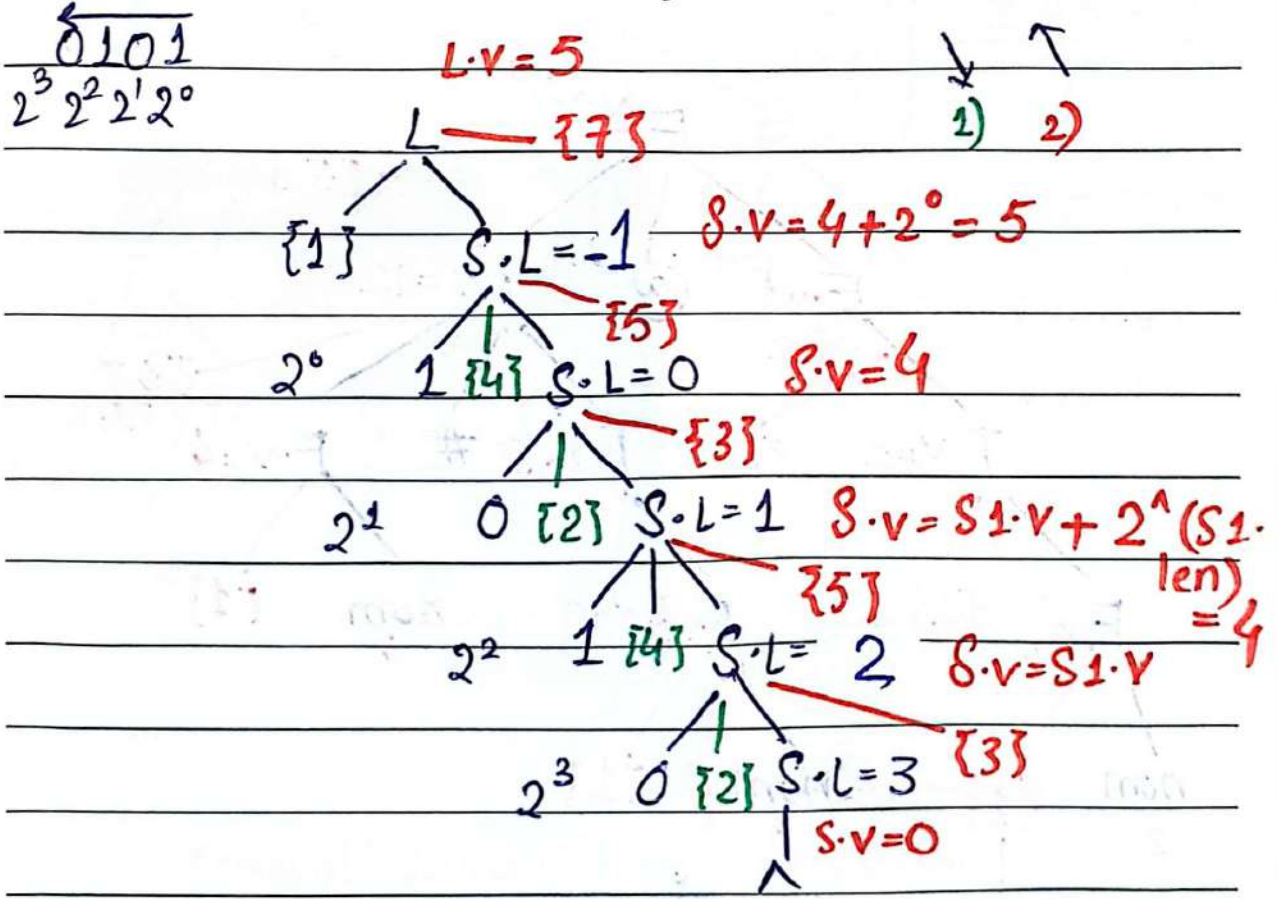
NO: _____
DATE: _____

①

$L \rightarrow \{S.len = 0\} S \{L.v = S.v\}$
 $S \rightarrow 0 \{S1.len = S.len + 1\} S1 \{S.v = S1.v\}$
 $S \rightarrow 1 \{S1.len = S.len + 1\} S1 \{S.v = S1.v\}$
 $S \rightarrow null \{S.len = 1\} \{S.v = 0\} + 2^1(S1.len)$
 $\{S.v = 0\}$

②

$S \rightarrow 0 S1 \{S.val = S1.val * 2\}$
 $S \rightarrow 1 S1 \{S.val = S1.val * 2 + 1\}$
 $S \rightarrow null \{S.val = 0\}$



can be left recursion

NO:

DATE:

8th April 2025

(SDT)

S-
attributed
grammar

Q#

Solve or
evaluate
exp

$2 @ 3 \# 4 \Rightarrow 14$

{print (F.v)}

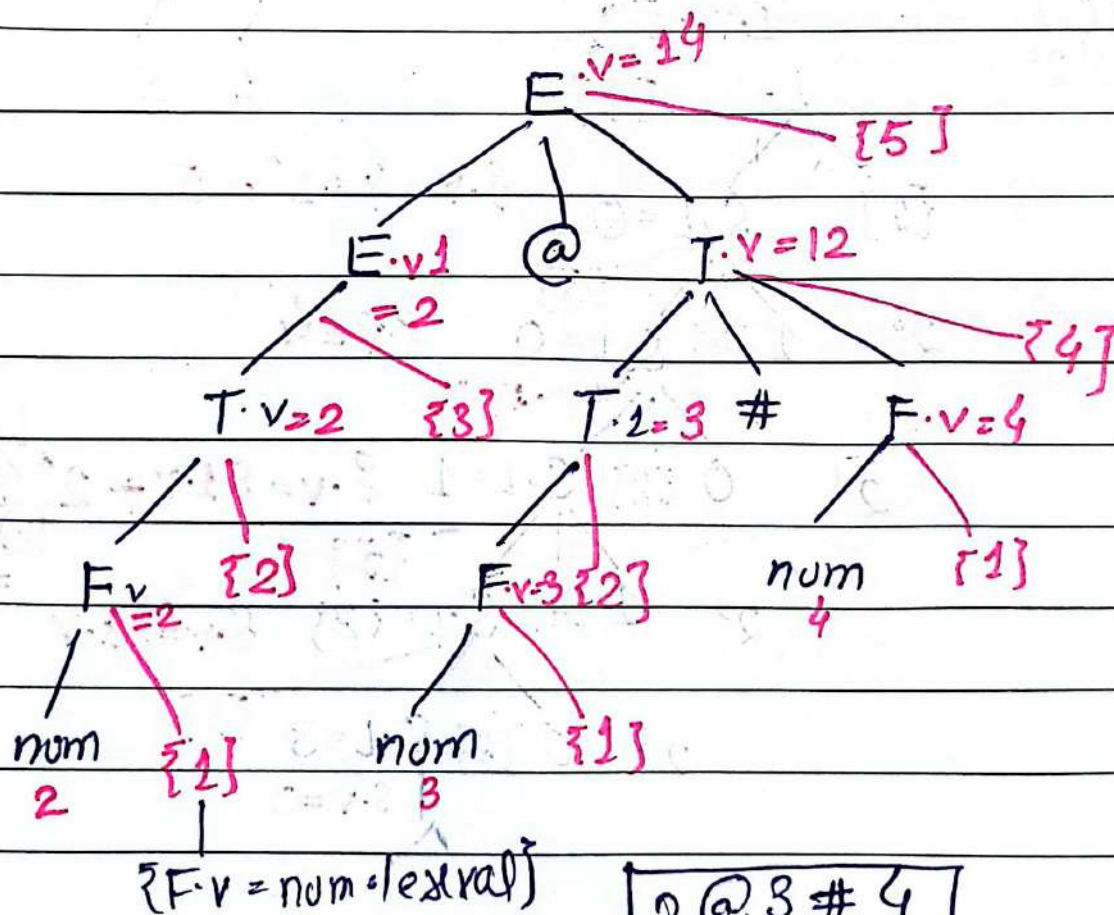
$E \rightarrow E_1 @ T \quad \{E.v = E_1.v + T.v\}$ (5)

$E \rightarrow T \quad \{E.v = T.v\}$ (3)

$T \rightarrow T_1 \# F \quad \{T.v = T_1.v * F.v\}$ (4)

$T \rightarrow F \quad \{T.v = F.v\}$ (2)

$F \rightarrow \text{num} \quad \{F.v = \text{num.lexval}\}$ (1)



localized

NO:

BDT

DATE:

(1)

$E \rightarrow E_1 @ T$

$\{ \text{print} (" + ") \}$

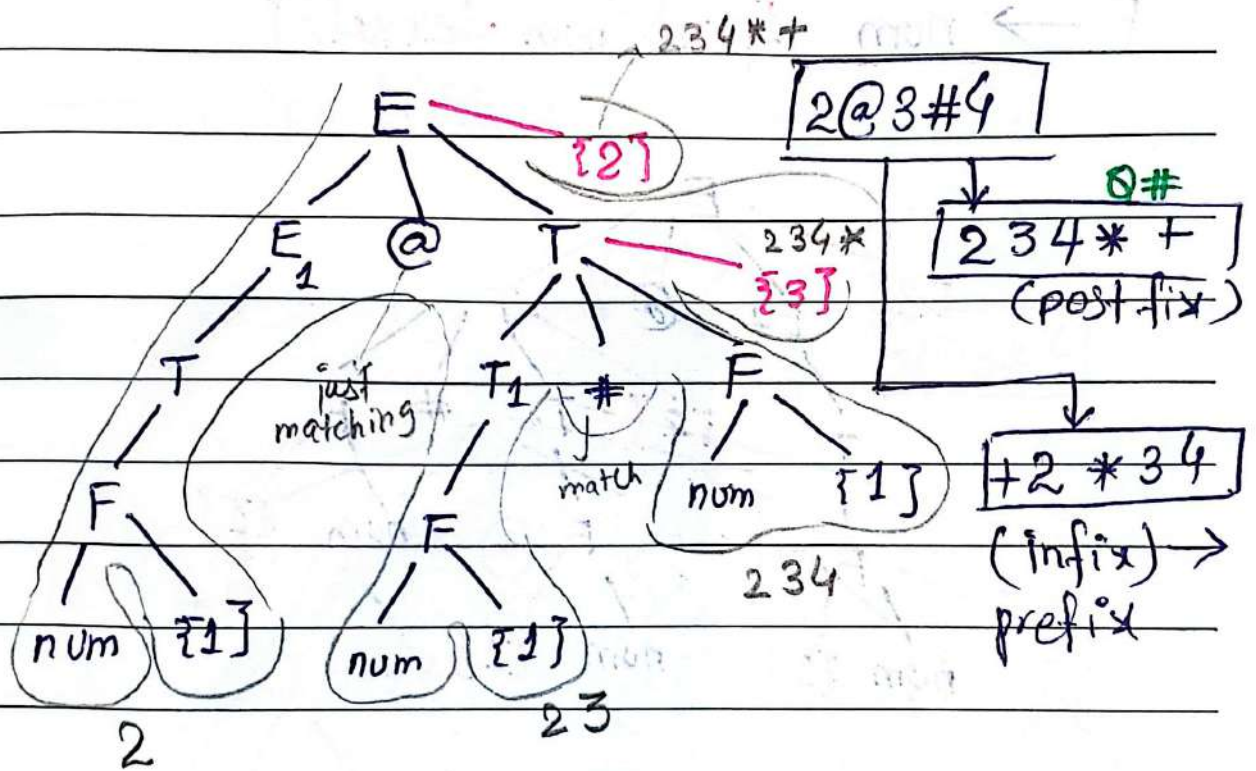
$E \rightarrow T^{(2)}$

$T \rightarrow T_1 \# F$

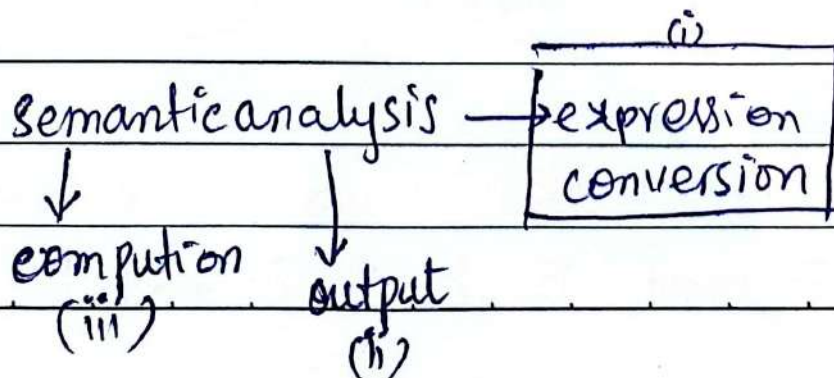
$\{ \text{print} (" * ") \}$

$T \rightarrow F^{(4)}$

$F \rightarrow \text{num}^{(5)} \{ \text{print} (\text{num} \cdot \text{lexval}) \}$



infix expression changed to postfix



NO:

DATE:

8# Infix to Prefix

for +2*34

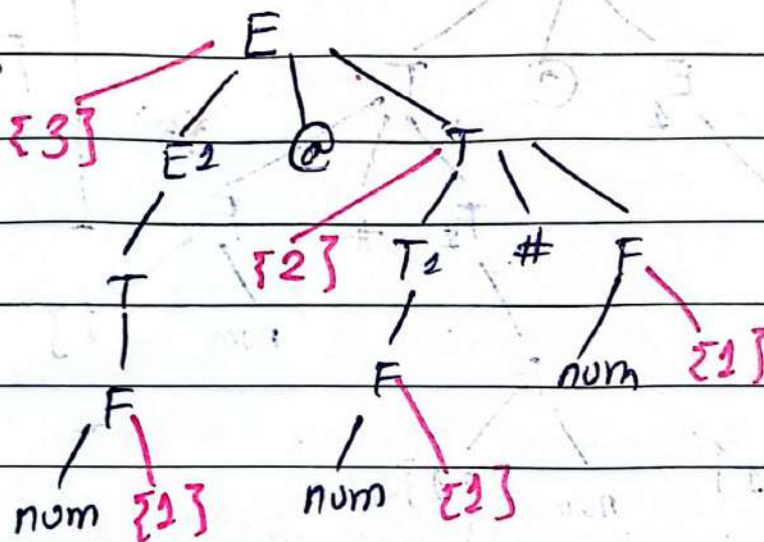
$E \rightarrow \{ \text{print} ("+") \} \quad E_1 @ T$ (3)

$E \rightarrow T$

$T \rightarrow \{ \text{print} ("*") \} \quad T_1 \# F$ (2)

$T \rightarrow F$

$F \rightarrow \text{num} \quad \{ \text{print} (\text{num} . \text{lexval}) \}$ (1)



$\Rightarrow$ +2*34

DryRun: Q#

NO:

DATE:

$E \rightarrow \{print("hi")\} E @ \{print("1")\} T \{print("oh")\}$

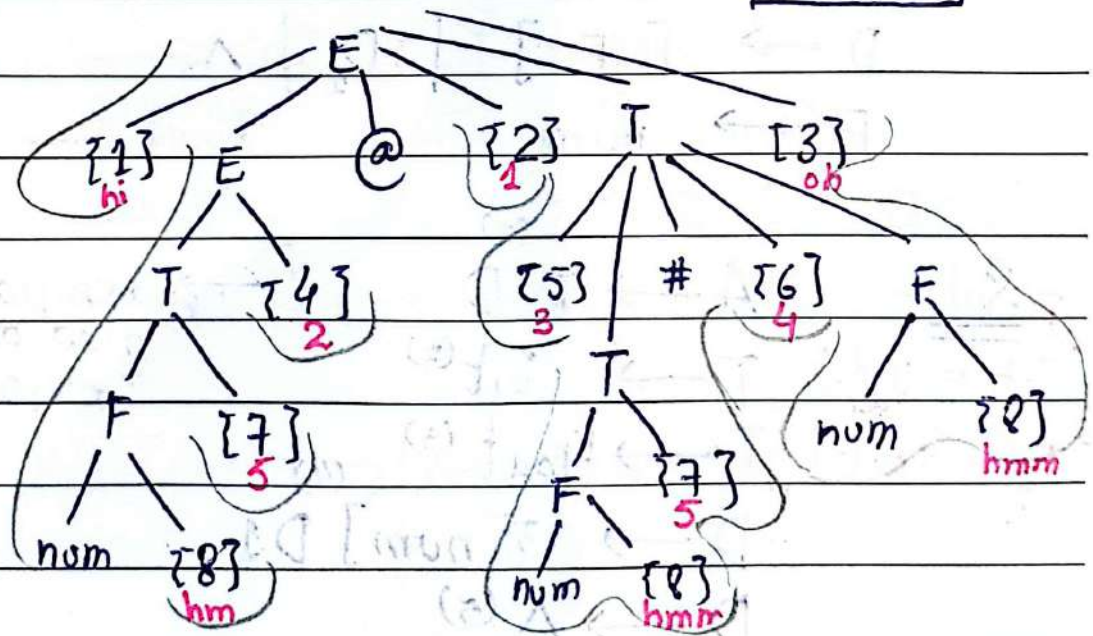
$E \rightarrow T \{print("2")\}$

$T \rightarrow \{print("3")\} T \# \{print("4")\} F$

$T \rightarrow F \{print("5")\}$

$F \rightarrow num \{print("hmm")\}$

2@3#4



Process $\rightarrow$ Top to Bottom $\rightarrow$ Left to Right

hi hmm 5 2 1 3 hmm 5 4 hmm oh

NO:

DATE:

Q# Array Type Expression :-

$\text{int}[2][3] \Rightarrow \text{array}(2; \text{array}(3, \text{integer}))$

① first write CFG ② write SDT

$E \rightarrow K [F] [F]$

$K \rightarrow \text{int} \mid \text{float}$

$F \rightarrow \text{num}$

$A \rightarrow K D$ (if dimensions variable)

$K \rightarrow \text{int} \mid \text{float}$

$D \rightarrow [F] \mid \wedge$

$F \rightarrow \text{num}$

Sol:-

$A \rightarrow T D$

$T \rightarrow \text{int}^{(2)}$

$T \rightarrow \text{float}^{(3)}$

$D \rightarrow [\text{num}] D_1^{(4)}$

$D \rightarrow \wedge^{(5)}$

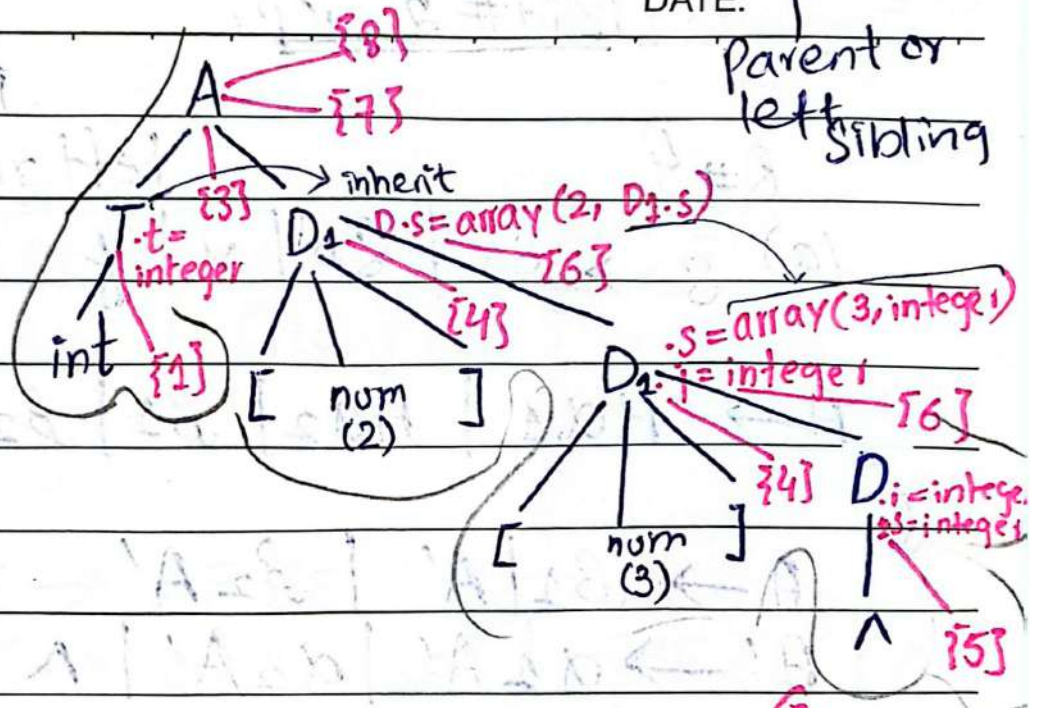
→ make parse tree on example

bottom-up $\rightarrow$ synthesis

bottom down $\leftarrow$ in herited
distributed.

NO:
DATE:

parent or
left sibling



$A \rightarrow T \quad \{ D.i = T.t \} \quad D \leftarrow \{ A.v = D.s \}$
 $T \rightarrow \text{int} \quad \{ T.t = \text{integer} \} \quad ①$
 $T \rightarrow \text{float} \quad \{ T.t = \text{float} \} \quad ②$
 $D \rightarrow [\text{num}] \quad \{ D2.i = D.i \} \quad ④ \quad D1$
 $D \rightarrow \wedge \quad \{ D.s = D.i \} \quad ⑤$

$\{ D.s = \text{array}(\text{num.lexval}, D1.s) \}$

L-attributed grammar

synthesis + inherited.

NO:

DATE:

Mid#2 prep:

mid #1

Fall 2020

Q#4

left recursion

$$(a) S \rightarrow Sa \mid Sb \mid x \mid y$$

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \beta_1 \mid \beta_2$$

$$A \rightarrow \beta_1 A' \mid \beta_2 A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \Lambda$$

$$\Rightarrow S \rightarrow xS' \mid yS'$$

$$S' \rightarrow aS' \mid bS' \mid \Lambda$$

$$(b) S \rightarrow E = : id$$

$$E \rightarrow num + E \mid num \mid S$$

$S \rightarrow E \rightarrow S$ (cycle) indirect
forward
substitution

$$S \rightarrow E = : id \text{ (no need)}$$

$$E \rightarrow num + E \mid num \mid E = : id$$

NO:

DATE:

mid #1

Q#3(b) left recursion

Sp-18

$$\begin{aligned}
 BE &\rightarrow \overset{A}{\boxed{BE}} \overset{d_2}{\text{and } BF} \mid \overset{A}{\boxed{BE}} \overset{d_2}{\text{or } BF} \mid \boxed{BF} \quad B \\
 BF &\rightarrow \text{true} \mid \text{false} \mid (BE)
 \end{aligned}$$

$$\begin{aligned}
 BE &\rightarrow BF BE' \\
 BE' &\rightarrow \text{and } BF BE' \mid \text{or } BF BE' \mid \Lambda \\
 BF &\rightarrow \text{true} \mid \text{false} \mid (BE)
 \end{aligned}$$

(c) left factor

indirect

back subs

$$S \rightarrow IF \mid ELSE$$

$$IF \rightarrow \boxed{\text{if } (E) S}$$

$$ELSE \rightarrow \boxed{\text{if } (E) S} \text{ else } S$$

$$S \rightarrow \boxed{\text{if } (E) S} \overset{\wedge B_2}{\uparrow} \mid \boxed{\text{if } (E) S} \overset{B_1}{\parallel} \text{ else } S$$

$$\begin{aligned}
 S &\rightarrow \text{if } (E) S \quad S' \\
 S' &\rightarrow \text{else } S \mid \Lambda
 \end{aligned}$$

NO:

DATE:

Sp 2020 Mid#1

Q#3(a)

CFG for $\{a^n b^n \mid n \geq 0\}$

$S \rightarrow a S b \mid \Lambda$

Fall 2016 Mid#1

not, and, or

Q#2(a)

$BE \rightarrow BE \text{ and } BE \mid BE \text{ or } BE \mid$
 $\text{not}(BE) \mid \text{true} \mid \text{false}$

$BE \rightarrow BE \text{ or } BF \mid BF$

$BF \rightarrow BF \text{ and } BG \mid BG$

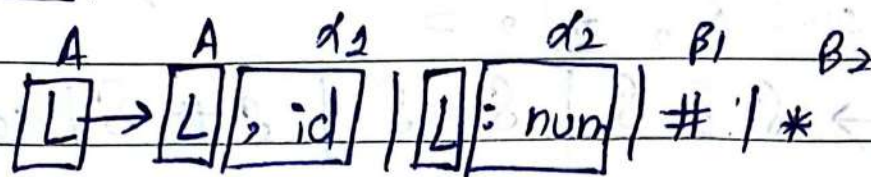
$BG \rightarrow \text{not}(BF) \mid BH$

$BH \rightarrow \text{true} \mid \text{false}$

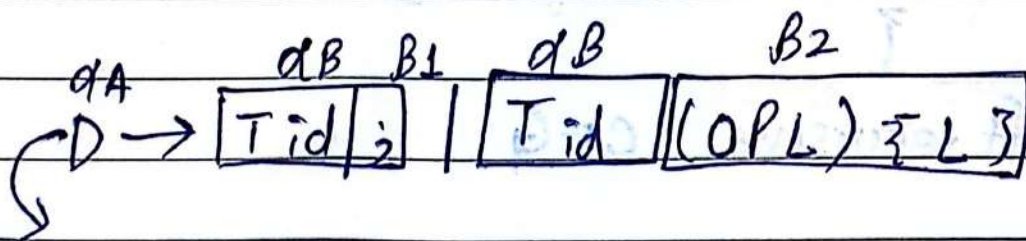
NO:

DATE:

Q#3(a): left recursion



$$L \rightarrow \# L' \mid * L'$$

$$L' \rightarrow , id L' \mid : num L' \mid \wedge$$
(b) $D \rightarrow VD \mid FD$ $VD \rightarrow T id ;$ $FD \rightarrow T id (OPL) \{L\}$ $OPL \rightarrow PL \mid \wedge$ $PL \rightarrow PL , T id \mid T id$  $D \rightarrow T id D'$ $D' \rightarrow ; \mid (OPL) \{L\}$ $OPL \rightarrow PL \mid \wedge$

α	β
PL	$PL , T id \mid T id$

 $PL \rightarrow T id PL'$ $PL' \rightarrow , T id PL' \mid \wedge$

NO:

DATE:

sp 2023 mid# 1:

Q#3: (left associative)

same $[8, \#], =$

$S \rightarrow S \& S \mid S = S \mid S \# S \mid id/num$

$S \rightarrow S \& A \mid S \# A \mid A$

$A \rightarrow A = F \mid F$

$F \rightarrow id \mid num$

Sp2023 Mid#2 CC

Q#1:

struct Student {

int id;

char grade;

}

left recursive CFG

$S \rightarrow \text{struct id } \{ E \}$

$E \rightarrow D \text{ id}; E \mid ;$

$E \rightarrow \wedge$

$D \rightarrow \text{int} \mid \text{char}$

NO:

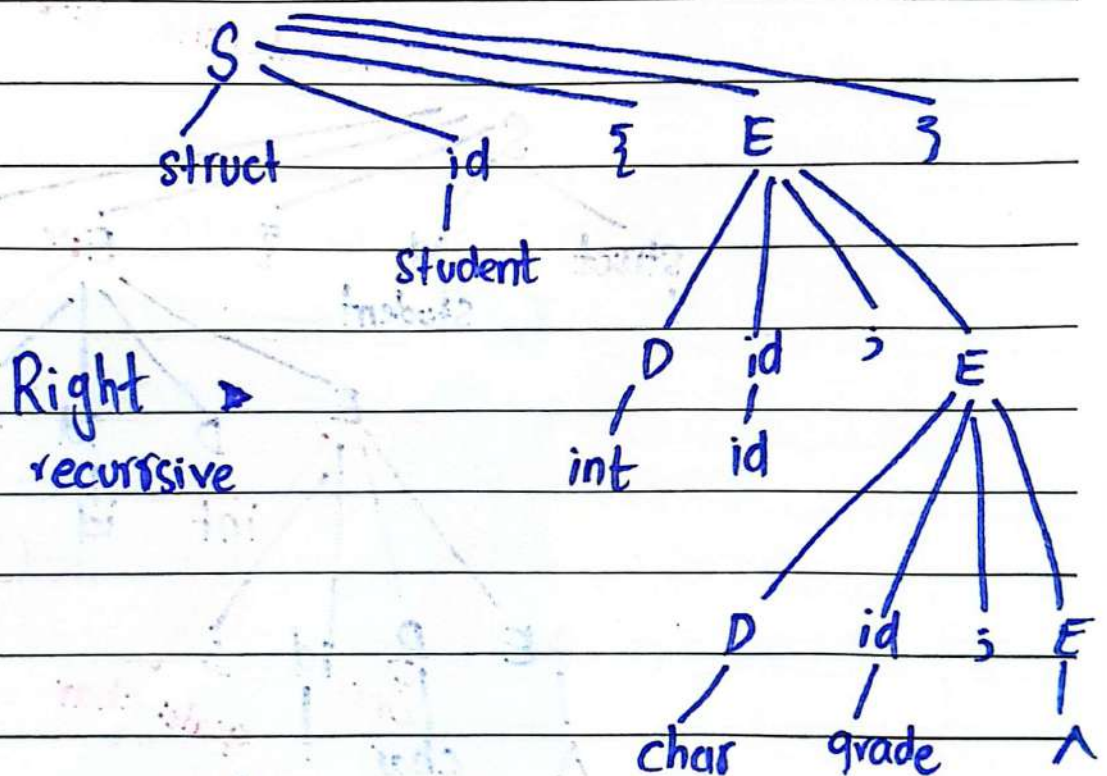
DATE:

$S \rightarrow \text{struct id } \{E\}$

$E \rightarrow D \text{ id } ; E$

$E \rightarrow \wedge$

$D \rightarrow \text{int} \mid \text{char}$



NO:

DATE:

Left Recursive

create table =

Student

$S \rightarrow \text{struct id } \{ E \}$

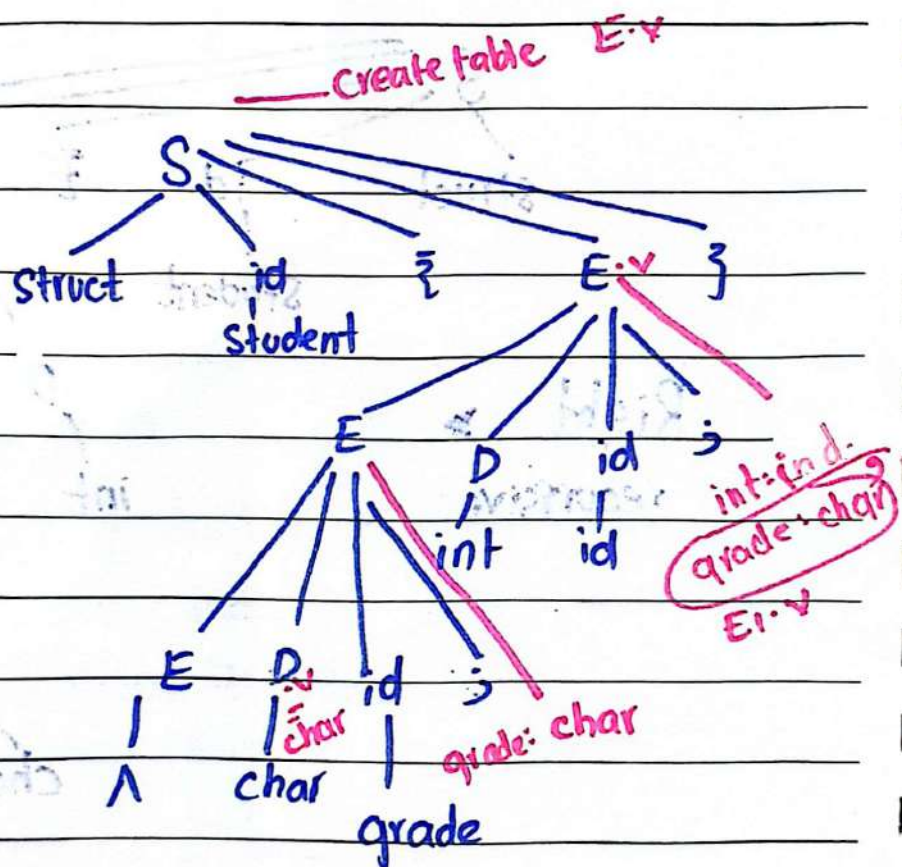
(id: int, grade: char)

$E \rightarrow E \text{ Did};$

$D \rightarrow \text{int}$

$D \rightarrow \text{char}$

$E \rightarrow \Lambda$



$S.v = \text{"create table"} + id.lex + "(" + E.v + ")"$

$S \rightarrow \text{struct id } \{ L \}$

$E \rightarrow E D id; \{ E.v = id.lex + ":" + D.v$

$D \rightarrow \text{int} \quad D.v = \text{int} + "," + E1.v \}$

$D \rightarrow \text{char} \quad D.v = \text{char}$

$E \rightarrow \wedge \quad \{ E.v = E.v \}$

Q#2: CFG:

$ROW \rightarrow R DL R'$

$DL \rightarrow DAT DL \mid \wedge$

tokens: $DAT \rightarrow D str D'$

$D' \rightarrow </td> \quad R \rightarrow <tx>$

$D \rightarrow <td> \quad R' \rightarrow </tx>$

$str \rightarrow \text{char}^*$

$\text{void ROW}() \{$

$\text{void DAT}() \{$

$\text{match}(R)$

$\text{match}(D)$

$DL();$

$\text{match}(str)$

$\text{match}(R');$

$\text{match}(D');$

$\}$

$\}$

$\text{void DL}() \{$

$\text{if} (look == D)$

$DAT();$

$DL();$

$\}$

else.

After Mid #2:

NO:

DATE:

15th April 2025

Semantic Analysis

↓ convert & compute.

Removing Left Recursion from a SDT

(non-left recursive grammar)

$$E \rightarrow E + T$$

$$E \rightarrow TE'$$

$$E \rightarrow T$$

$$E' \rightarrow +TE'$$

$$T \rightarrow \text{num}$$

$$E' \rightarrow \Lambda$$

$$T \rightarrow \text{num}$$

$$A \rightarrow A\alpha / \beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' / \Lambda$$

e.g infix to postfix conversion

$$E \rightarrow E_1 [+T] \text{ print } ("+")$$

$$E \rightarrow T$$

$$T \rightarrow \text{num} \text{ print } (\text{num} \cdot \text{lexval})$$

$$2 + 3 \Rightarrow 23 +$$

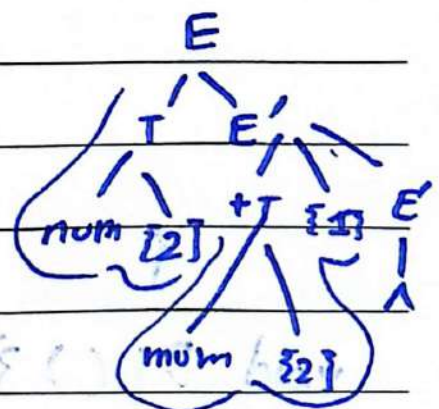
②

$$E \rightarrow TE'$$

$$E' \rightarrow + (T [1]) E'$$

$$E' \rightarrow \Lambda$$

$$T \rightarrow \text{num} [2]$$



23+

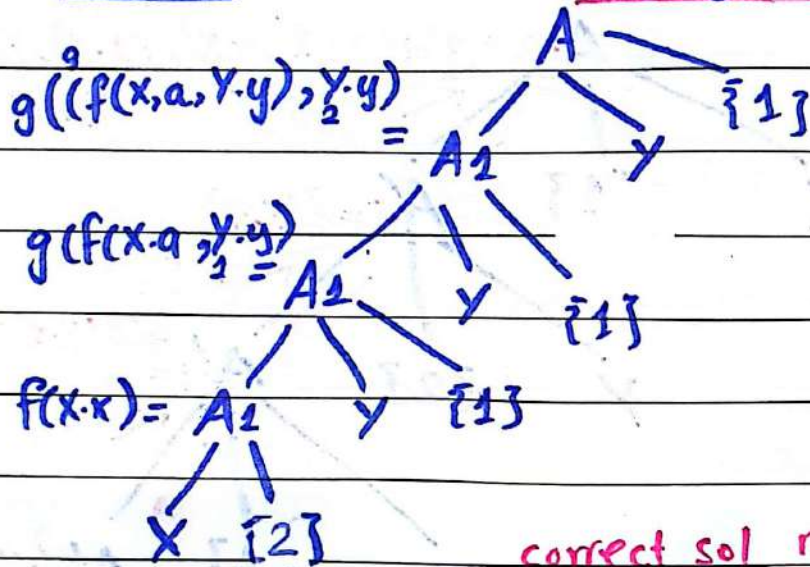
computation:

$$A \rightarrow A_1 Y \quad \{A \cdot a = g(A_1 \cdot a, y \cdot y)\}$$

$$A \rightarrow X \quad \{A \cdot a = f(x \cdot x)\}$$

 $\Rightarrow$ input string:Xyyy

$$A \cdot a = g(g(g(f(x, a, y \cdot y), y \cdot y), y \cdot y), y \cdot y)$$

correct sol next page $\rightarrow$

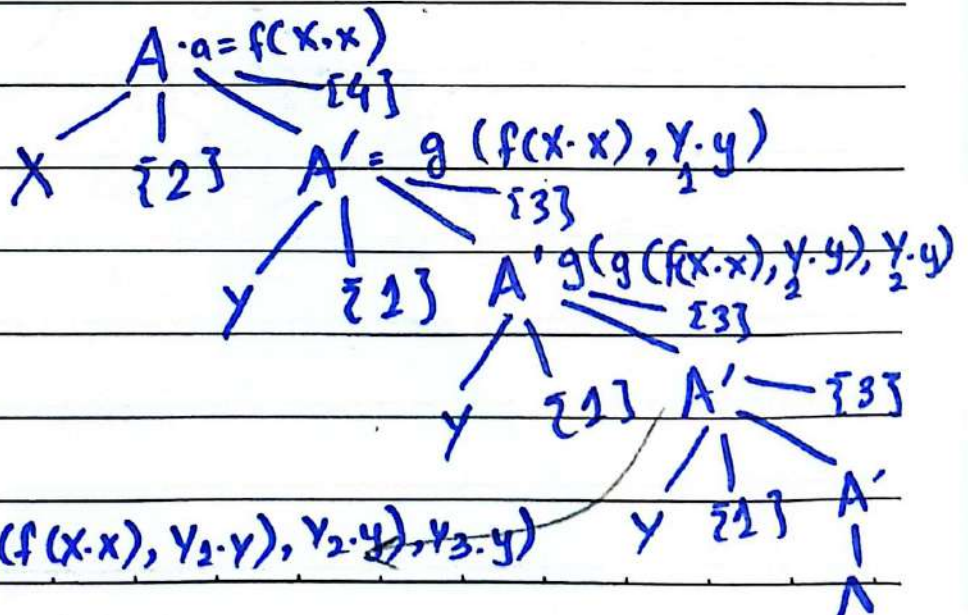
②

④

$$A_1 \rightarrow X \quad \{A \cdot a = f(x \cdot x)\} \quad A' \quad \{A_1 \cdot a = A' \cdot a\}$$

$$A' \rightarrow Y \quad \{A' \cdot a = g(A_1 \cdot a, y \cdot y)\} \quad A' \quad \{A' \cdot a = A' \cdot a\}$$

$$A' \rightarrow \Lambda$$



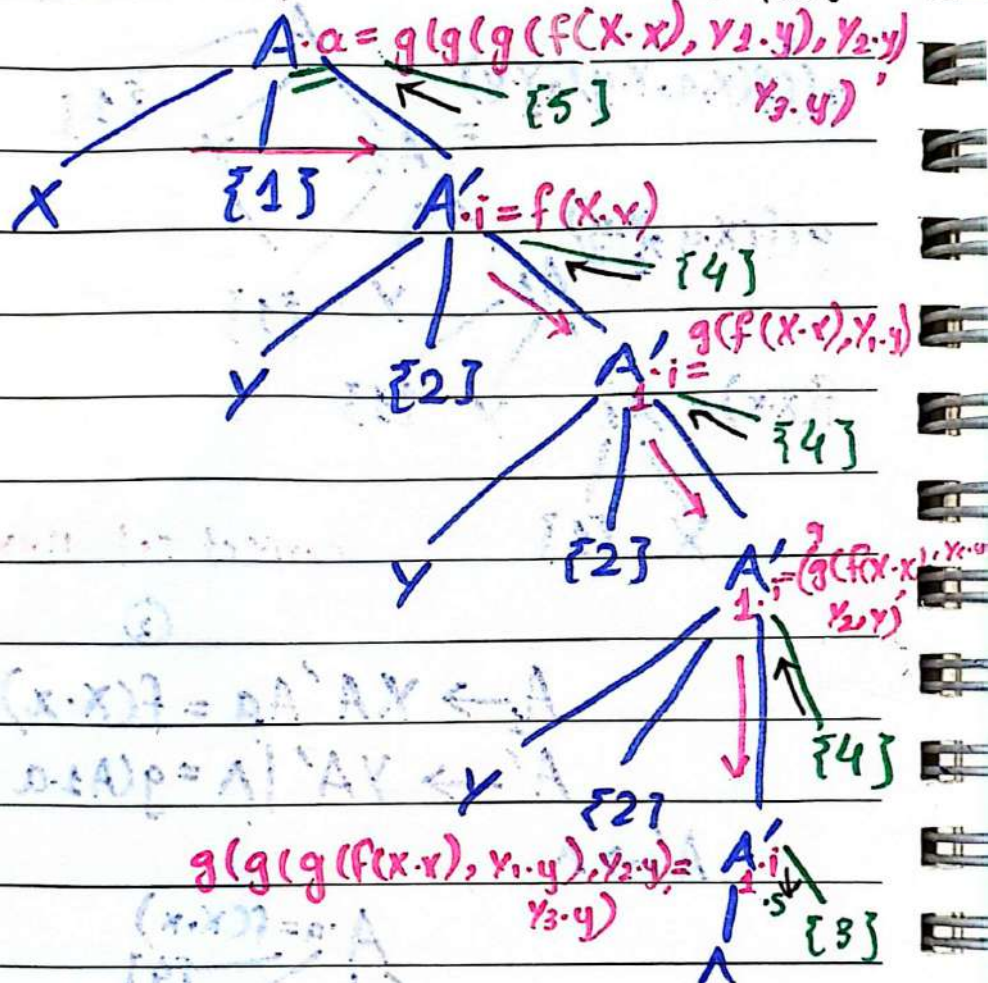
NO:

DATE:

$A \rightarrow X \{A'.i = f(X.x)\} A' \{A.a = A'.s\}$ ④
 $A' \rightarrow Y \{A'.i = g(A'.i, Y.y)\} A' \{A'.s = A'.s\}$
 $A' \rightarrow \Lambda \{A'.s = A'.i\}$ ③

if $A'.s = A'.i$
 only below
 level copy
 not last

XYYY



$A'.s = A'.i$

$g(g(g(f(X.x), y_1.y), y_2.y), y_3.y)$
 $y_3.y$

NO:

DATE:

$$2 + 3 = \textcircled{5}$$

$$E \rightarrow E1 + T \quad \{E.v = E1.v + T.v\} \textcircled{1}$$

$$E \rightarrow T \quad \{E.v = T.v\} \textcircled{2}$$

$$T \rightarrow \text{num} \quad \{T.v = \text{num.lexval}\} \textcircled{3}$$

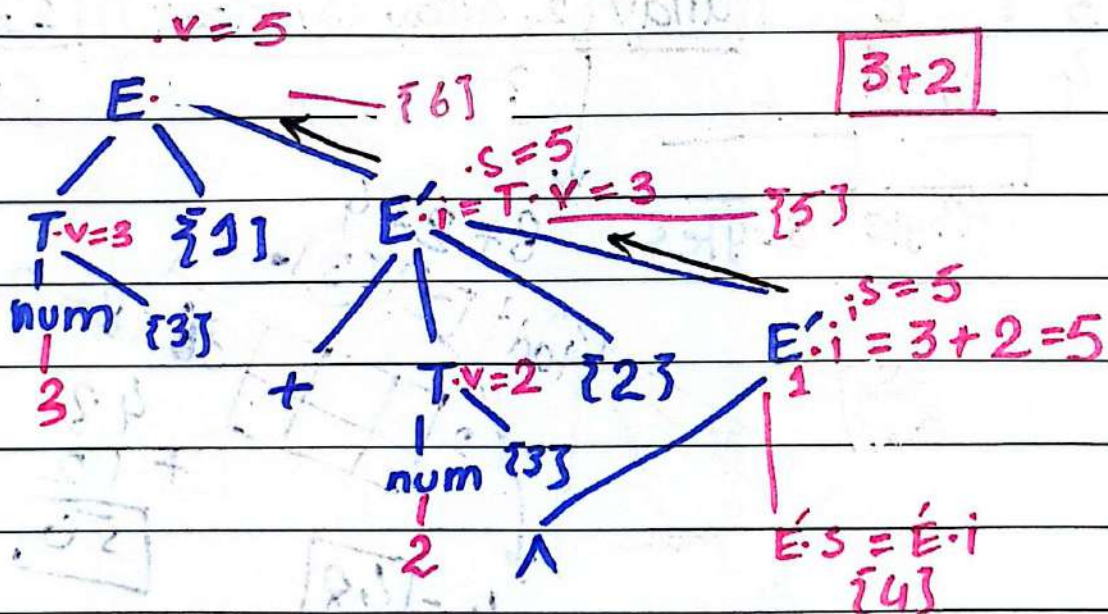
$$T \rightarrow \text{id} \quad \{T.v = \text{num.lexval}\} \textcircled{4}$$

$$E \rightarrow T \quad \{E.i = T.v\} \quad E' \quad \{E.v = E'.s\} \textcircled{1}$$

$$E' \rightarrow + T \quad \{E'.i = E'.i + T.v\} \quad E' \quad \{E'.s = E'.s\} \textcircled{2}$$

$$E' \rightarrow \wedge \quad \{E'.s = E'.i\} \textcircled{4}$$

$$T \rightarrow \text{num} \quad \{T.v = \text{num.lexval}\} \textcircled{3}$$



NO:

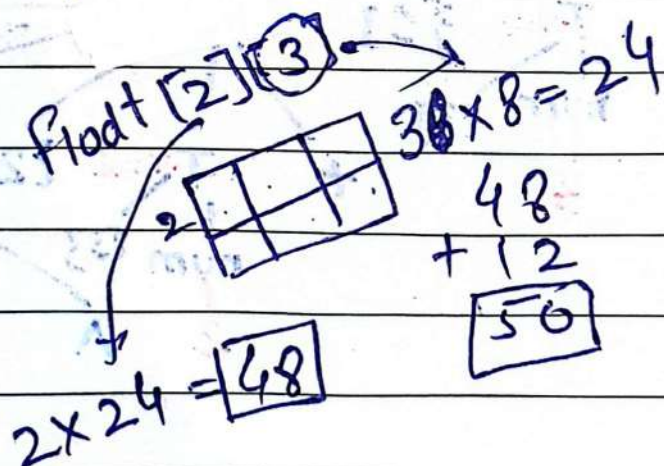
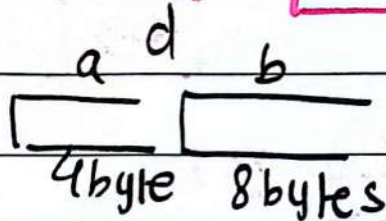
DATE:

17 April 2025 SDT Phase #3

*.cpp

```
int a;  
float b;  
float [2][3];
```

pos	lexeme	Type	offset
1	a	integer (4)	0
2	b	float (8)	4
3	c	array (2, array (3, float))	12
4	d	?	60



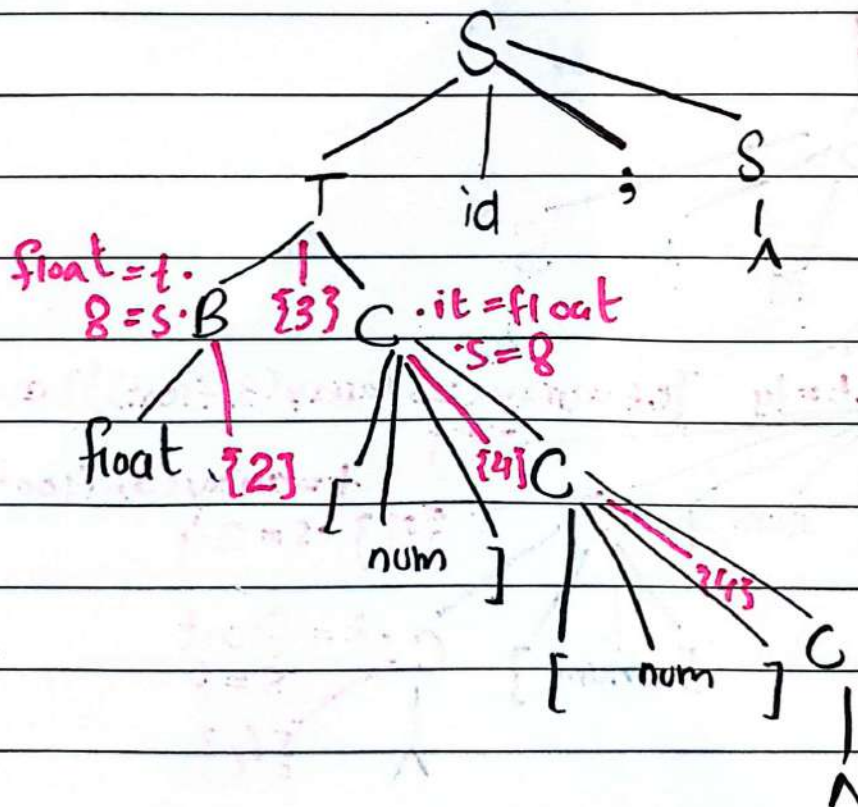
Problem.

NO:

DATE:

- 1 $\boxed{S} \rightarrow \wedge$ SV
- 2 $S \rightarrow T \text{ id} ; S1$
- 3 $T \rightarrow B$ ③ $\{C.it = B.t ; C.is = B.s\}C$
- 4 $B \rightarrow \text{int}$ ① $\{B.t = \text{integer} ; B.s = 4\}$
- 5 $B \rightarrow \text{float}$ ② $\{B.t = \text{float} ; B.s = 8\}$
- 6 $C \rightarrow [num]$ ④ $\{C1.it = C.it ; C$
- 7 $C \rightarrow \wedge$ ④ $C.is = C.is\}$

using $\Rightarrow$
local
variable



DATE: 8/1

↓ not
Attached
with any in CFG

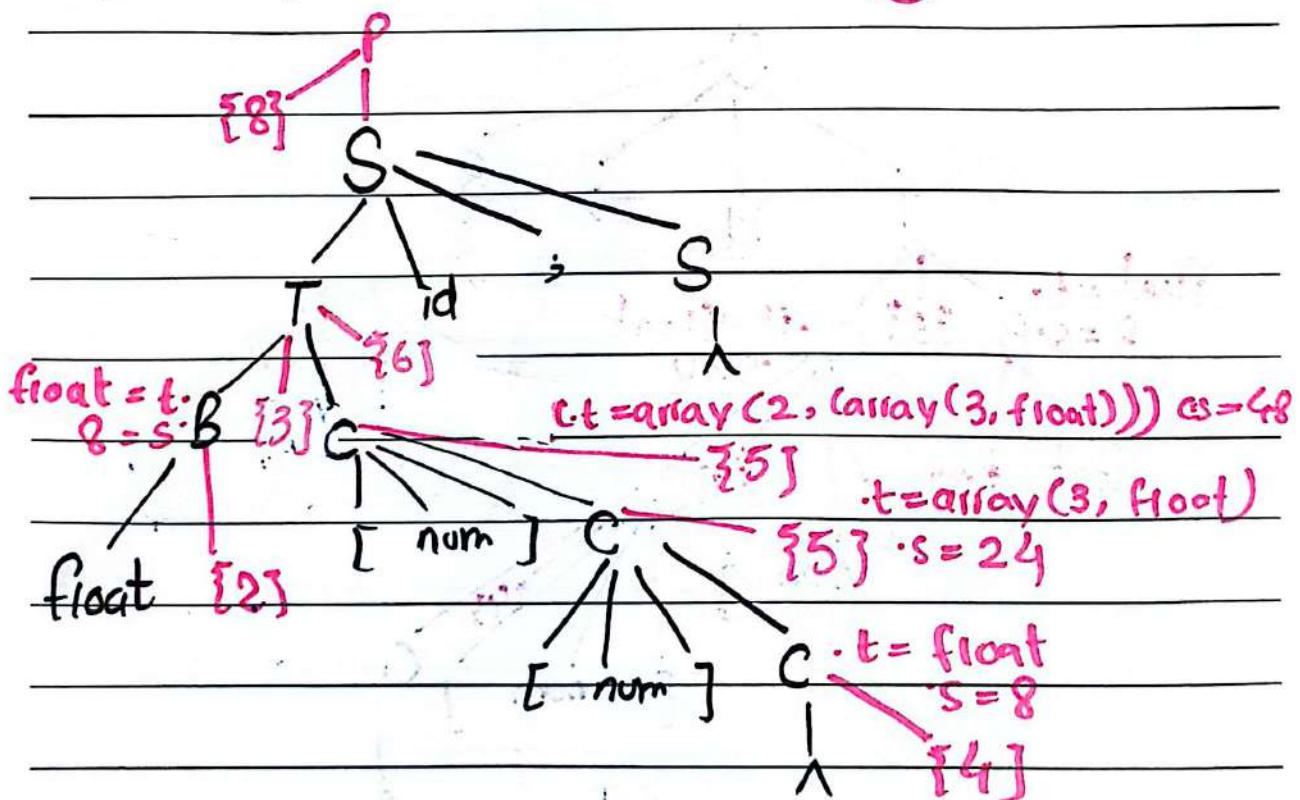
1 $S \rightarrow \wedge$

2. $S \rightarrow T \text{ id} ; S1$

4 $B \rightarrow \text{int} \quad [B.t = \text{integer} \Rightarrow B.s = 4]$

6 $C \rightarrow [\text{num}] C \{5\} \textcircled{2}$

7 $C \rightarrow \wedge$ [$C.t = \text{type}$; $C.s = \text{size}$]
(4)



5: $C.S = C1.S * \text{num.lexval};$

C: $t = \text{array}(\text{num. lexical}, C1.\text{type})$

6: $T.type = c.t$; $T.size = c.s$;

top $\uparrow$ lookup top.get

NO: _____

DATE: _____

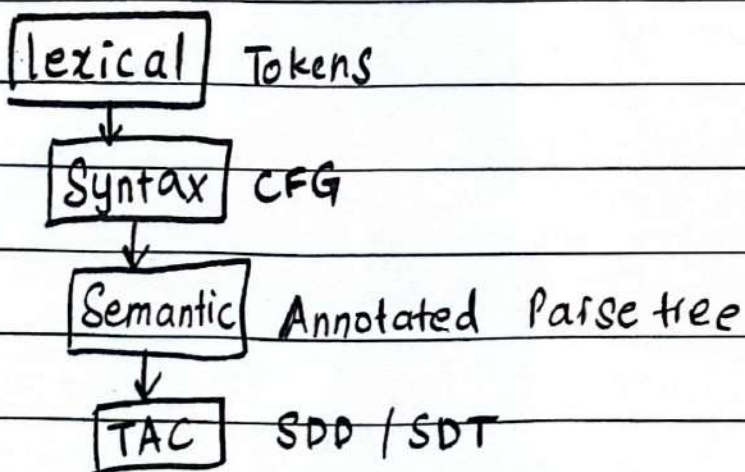
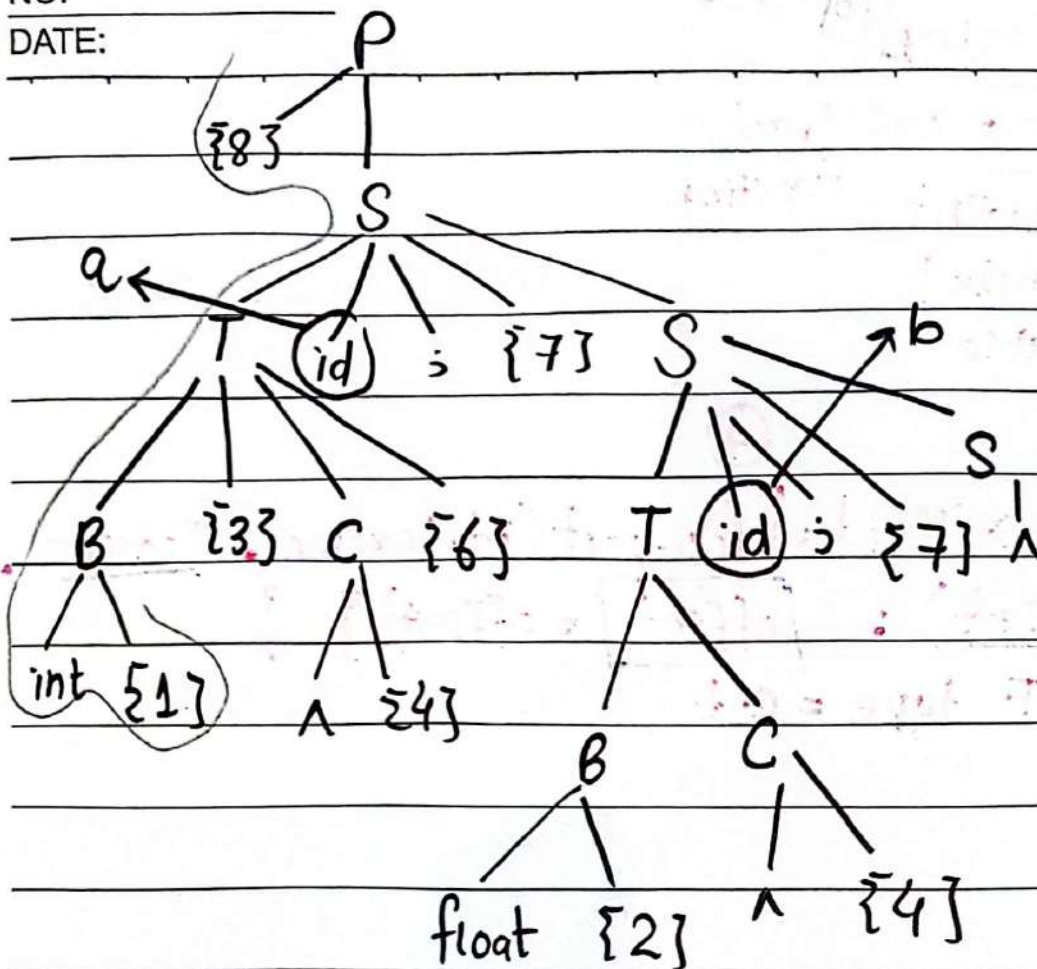
• put $\rightarrow$ abstract
current function
symbol
table

⑦

$S \rightarrow T : id ; \{ \text{top.put}(id.lexeme, \text{T.type},$
 $\text{offset}) ; \text{offset} = T.size \}$

NO:

DATE:



22nd April 2025

Three Address Code

Statements:-

1) Assignment :

1.1 $x = y \text{ op } z$

1.2 $x = \text{op } z$

2) Copy :

2.1 $x = y$

3) Jump :

3.1 Unconditional $\nearrow$ label

3.1.1 goto L / line #

3.2 Conditional

3.2.1 if x goto L / line #

3.2.2 if $x \text{ rel-op } y$ goto
L / line #

4) Array : $a[i] = b[j]$

$x = b[j]$

$a[i] = x$

5) Function :

add (a, b)

param a;

param b;

call add, 2

NO: _____

DATE: _____

6) Pointer:

$$x = *y$$
$$x = \&y$$

Q#1: $x = a + b + c * d$

$$t1 = c * d$$

$$t2 = a + b$$

$$t3 = t2 + t1$$

$$x = t3$$

Q#2: $d = c + a[c][j]$

next x $t1 = a[i][j]$

page

$$t2 = c + t1$$

$$d = t2$$

NO:

DATE:

2D array $\rightarrow$ in memory $\rightarrow$ consecutive locations

int[2][3]

$a[i][j]$

	0	1	2
0	— 0	— 1	— 2
1		11	

$$t1 = i \times 3$$

$$= i \times \# \text{ of cols.} \times \text{size}$$

$$t1 = i \times (3 \times 4) \quad (\text{rows})$$

skip
3
cells

int = 4 bytes

$$t2 = j \times 4 \quad (\text{columns})$$

$$t3 = t1 + t2$$

$$t4 = a[t3]$$

— That's how you deal with 2D array.

$$t5 = c + t4$$

$$d = t5$$

e.g #1.

+

- → signed numbers

CFG:

1 $S \rightarrow id = E$;

2 $E \rightarrow E_1 + E_2$

3 $E \rightarrow -E_1$

4 $E \rightarrow id$

IAC: $a = b + (-c)$ transforms into
 binary unary & n

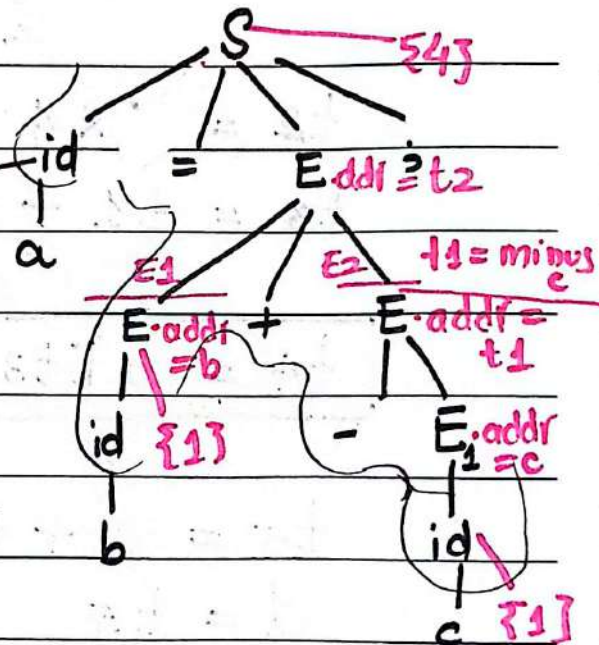
$t1 = \text{minus } c$

$t2 = b + t1$

$a = t2$

next page
 →

We will get this from symbol table

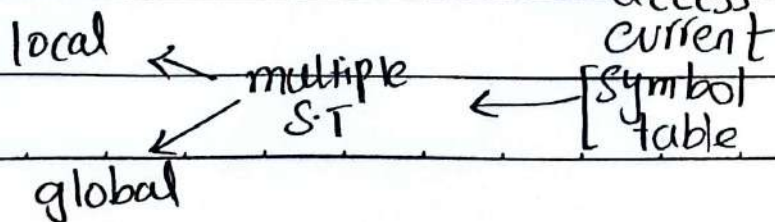


1

2

3 $E \rightarrow -E_1$ $\{ E.addr = \text{new Temp}();$

4 $E \rightarrow id$ $\{ E.addr = \text{top.get}(id.lexem - c); \}$



emit / gen print line

NO: _____

DATE: _____

1 $S \rightarrow id = E;$

(4) { gen (top.get (id.lexeme)) '=' E.addr }

$E \rightarrow$ (3) $\rightarrow +2$
2 $E1 + E2$ { E.addr = new Temp();
gen (E.addr '=' E1.addr + E2.addr) }

3 $E \rightarrow - E1$ (2) $\rightarrow +1$
{ E.addr = new Temp();
gen (E.addr '=' 'minus' E1.addr); }

4 $E \rightarrow id$ { E.addr = top.get (id.lexeme); }
(1)

pg #383 Ullman

eg #2: $d = c + a[c][j]$

1 $S \rightarrow id = E;$

2 $E \rightarrow E1 + E2$

3 $E \rightarrow L$

4 $L \rightarrow L1 [E]$ can be any dimension-
array

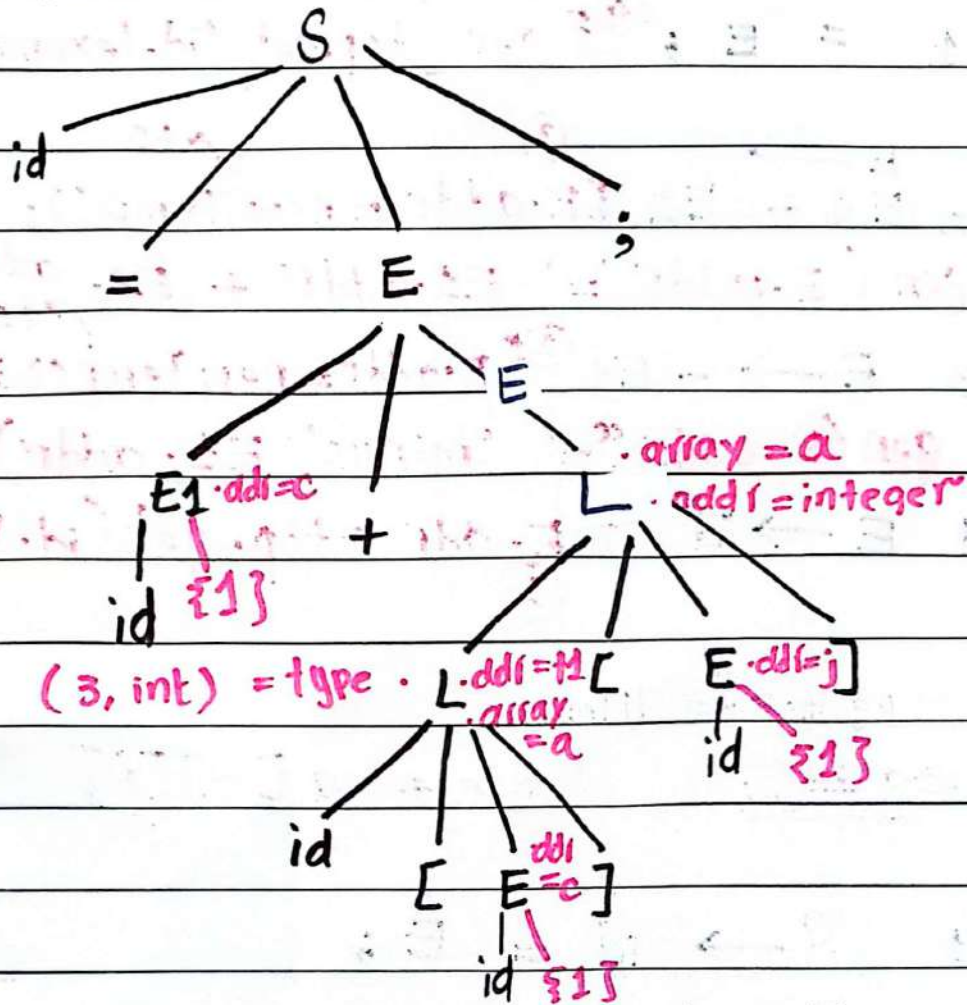
5 $E \rightarrow id$

6 $L \rightarrow id [E]$

NO: _____

DATE: _____

element type
a type = array(3, array(2, int))



1

2

next page →

3

4 $E \rightarrow id$ $\{E.addr = top.get(id.lexeme); \}$

$t_2 = j * 4$

5 $L \rightarrow L1[E]$ $L.array = L1.array$
 $L.type = L1.type.array.element$

$t = new Temp()$
 $gen(t, '=', E.addr, '*', L.array, \sqrt{t+1}, width); gen($

6 $L \rightarrow id [E]$ $\{ L.addr = new Temp(); L.array = top.get(id.lexeme);$
 $L.type = L.array.type.element; \}$

gen (L.addr ' = ' E.addr ' * '
L.array.width) } $\rightarrow t1 = i * 12$
 $3 \times 4 = 12$

24th April 2025

3AC: *ans*

Pg # 383:

Input :- $d = c + a[i][j]$

385,
Ullman

int a[2][3] Output :-

	0	1	2
0			
1			

$t1 = i * \# \text{ of cols} * \# \text{ of bytes.}$

$t2 = j * 4$

$t3 = t1 + t2$

$t4 = a[t3]$

$t5 = c + t4$

$\Rightarrow d = t5$

e.g. $a[0][1]$

1 $S \rightarrow id = E ;$ $t1 = 0 \times 3 \times 4$

2 $E \rightarrow E1 + E2$ $t1 = 0$

3 $E \rightarrow id$ $t2 = 1 \times 4$

4 $E \rightarrow L$ $t2 = 4$

5 $L \rightarrow L1[E]$ $t3 = 0 + 4 = 4$

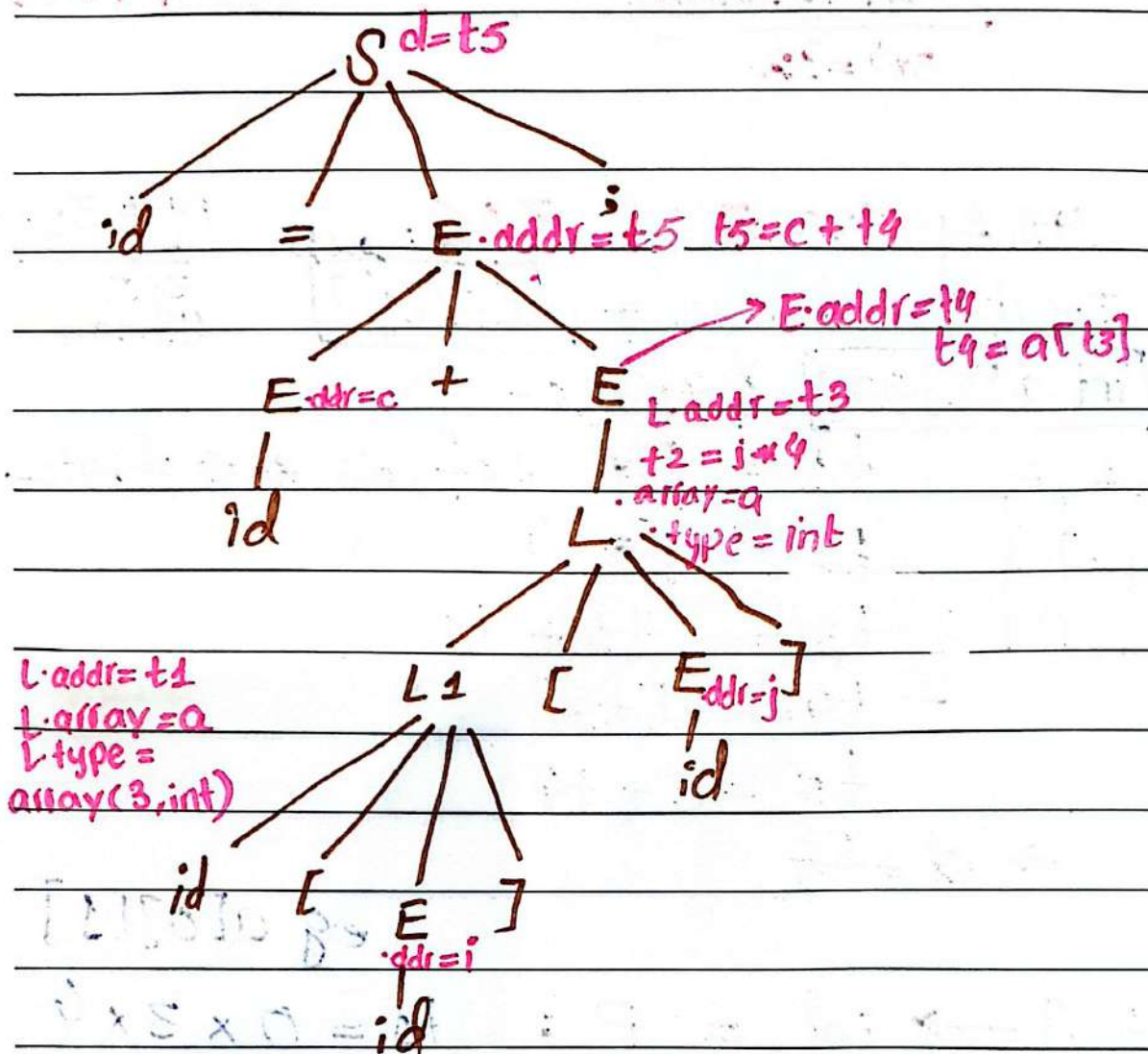
6 $L \rightarrow id[E]$ $a[t3]$

$\downarrow 4 \text{ bytes.}$

	0	1	2
0	0	4	8
1	12	16	20

NO:

DATE:



pos	lexeme	type
1st	a	array(2, array(3, int))

$a.type$

2) $E \rightarrow E_1 + E_2$ { $E.addr = \text{new Temp}()$;

3) $E \rightarrow id \quad [E.addr = \text{topget}(id.lexeme)]$

4) $E \rightarrow L \{ E.addr = newTemp();$

5) $L \rightarrow L1[E]$ { $L.array = L1.array$; $L.type = \underline{L1.type}$.
element ; $L.type.width = 4$; $t = \text{new Temp}()$;

g) $L \rightarrow id[E] \{ L.addr = new Temp(); L.array = Top.get(id.lexeme); L.type = L.array.type.element; \underline{L.type.width = 12}; gen(L.addr = E.addr, '* L.type.width); \}$

↓
function
= 3 × 4

```
→ gen(L.addr '=' L1.addr '+' t); }
```

right to left associativity $\leftarrow (!) > \&\& > \parallel \rightarrow$ left to right associativity.

NO:

DATE:

3AC :-

pg #404: Ullman

↑ jump

if ($a < b \parallel c > d \&\& e \neq f$)

↓
boolean
expression
in it

boolean
operators = ' $\parallel$ ' ' $\&\&$ '

$B \rightarrow B_1 \parallel B_2$

$B \rightarrow B_1 \&\& B_2$

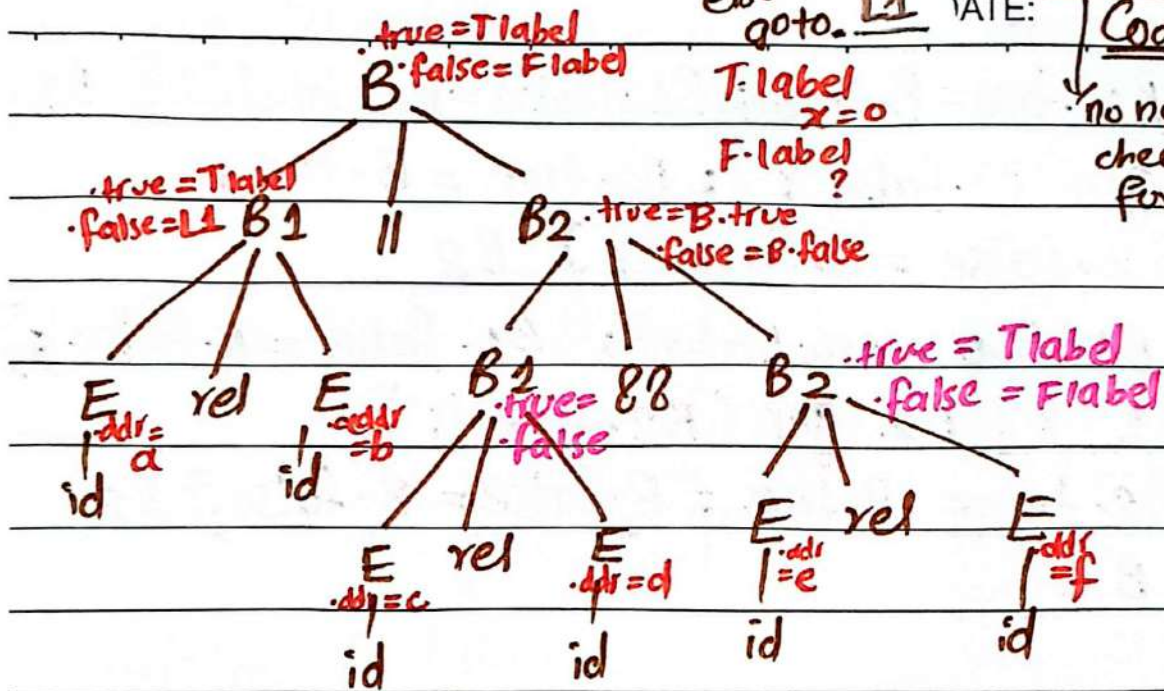
$B \rightarrow ! B_1$

$B \rightarrow E \text{ (rel) } E$ $\rightarrow \text{rel.op}$

$E \rightarrow id$

⇒ and precedence greater so in depth.

if $a < b$
 then goto Tlabel
 else goto L1
 NO: Short Circuit
 IATE: Code :-
 no need to check further



L1 : if $c > d$ goto L2

else Flabel

L2 : if $e \neq f$ goto Tlabel

else Flabel

NO:

DATE:

- 1) $\{ B1.true = B.true ; B1.false = \text{new label}() \}$ B1
 $\parallel \text{ gen}(B1.false ':') ; B2.true = B.true ;$
 $B2.false = B.false ; \}$ B2
- 2) $\{ B1.true = \text{new Label} ; B1.false = B.false ; \}$
 $B1 \parallel \text{ gen}(B2.true ':') ;$
 $B2.true = B.true ; B2.false = B.false ; \}$ B2
- 3) ! B1
- 4) $E1 \text{ rel } E2 \{ \text{gen}('if' E1.addr \text{ rel op } E2.addr 'goto'$
 $\boxed{B.true} ; \text{gen}(\text{goto } \boxed{B.false}) ; \}$
- 5) $E \rightarrow id \{ E.addr = \text{Top.get}(id.lexeme) ; \}$

code:

while (a < b) {

if (c != d) {

x = y + z

}

else

x = y - z

}

29th April 2025

NO:

DATE:

while loop.

8) $S \rightarrow \{ S.begin = \text{new label}; \}$ $\xrightarrow{L1}$ $S.next = \text{new label}; \}$ $\xrightarrow{END}$ $gen(S.begin ':') ; \}$

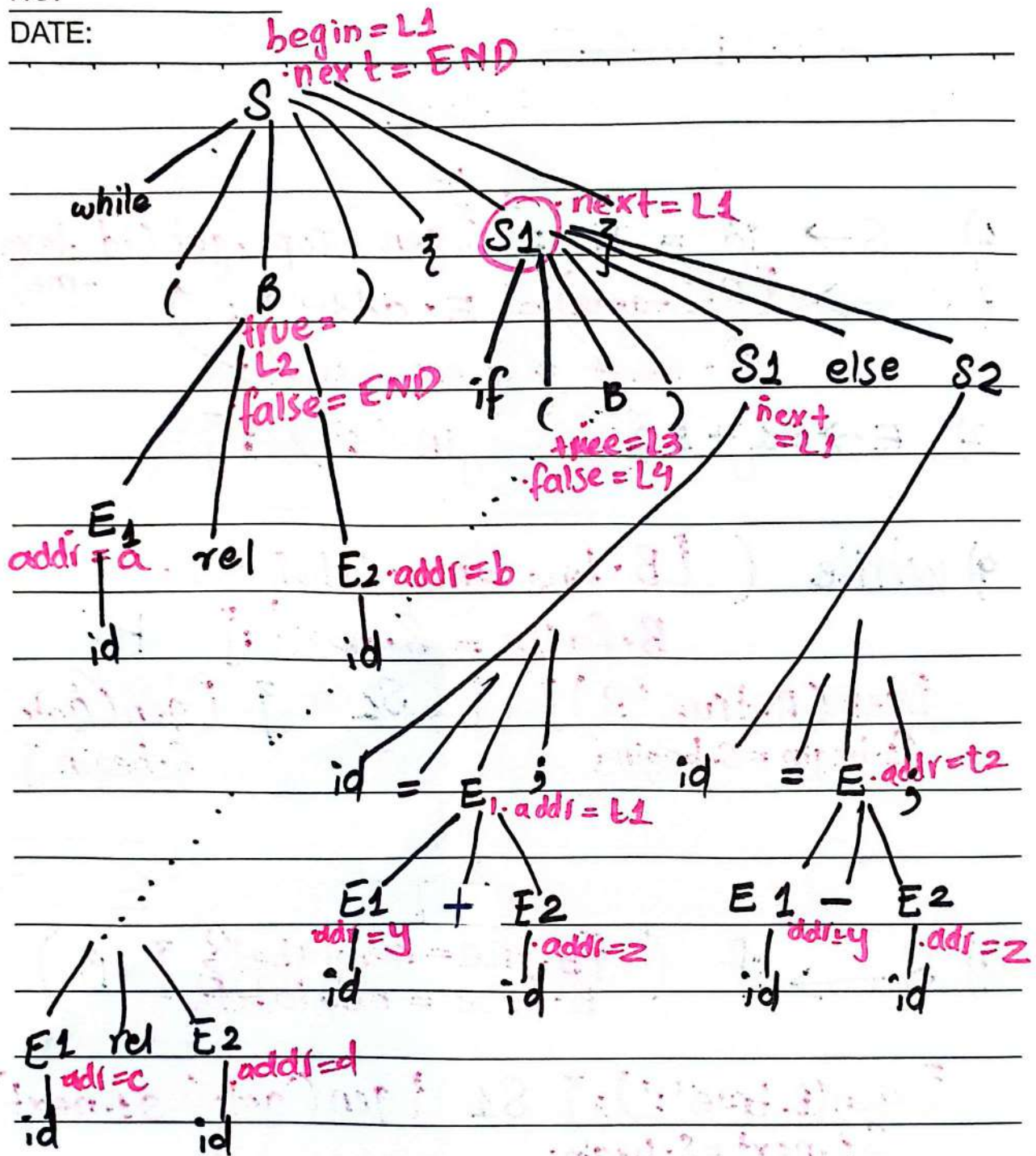
while ($\{ B.true = \text{new label}(); \}$ $B.false = S.next; \}$ B)
 $\{ gen(B.true ':') ; \}$ $S1 \}$ $\{ gen('goto' S1.begin) ; \}$
 $S2.begin = S.begin ;$

9) $S \rightarrow \text{if} (\{ B.true = \text{new label}(); \} B)$
 $\{ gen(B.true ':') ; \}$ $S1 \}$ $\{ gen('goto' S1.next) ; \}$
 $S1.next = S.next$

else $S2$
 $\{ gen(B.false ':') ; \}$

NO:

DATE:



$B \rightarrow E_1 \text{ rel } E_2$

$\{ \text{gen (if } E.\text{addr} \text{ rel op } E_2.\text{addr} \text{ 'goto' } B.\text{true}) \text{ gen ('goto' } B.\text{false}) \}$

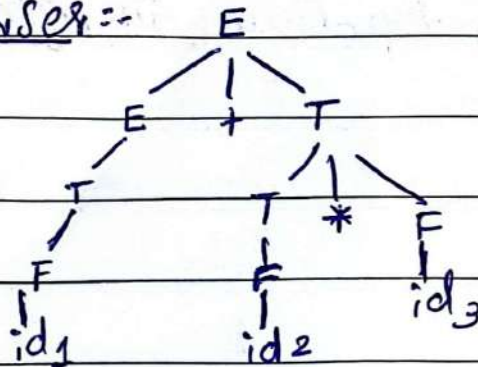
6th May 2025

$E \rightarrow E + T \quad | \quad T$
 $T \rightarrow T * F \quad | \quad F$
 $F \rightarrow \text{id}$

reduce $\rightarrow$ mapping
 find out & replace

$\text{id}_1 + \text{id}_2 * \text{id}_3$

Topdown parser:-



$\Rightarrow$ Rightmost derivation Tree

Bottomup parser: start with input string, find out
 $\text{id}_1 + \text{id}_2 * \text{id}_3$ mapping $\rightarrow$ look at rules right side
 $F + \text{id}_2 * \text{id}_3$ and map at left

$T + \text{id}_2 * \text{id}_3$

$E + \boxed{T * F}$

$E + \text{id}_2 * \text{id}_3$

$E + T \Rightarrow \textcircled{E}$ start variable
 input payable
 from grammar

$\text{id} \quad E + F * \text{id}_3$

$\text{id} \quad E + \textcircled{T} * \text{id}_3$

can reduce but not valid because no
 rule like this.

NO: Handle Pruning :-
 DATE: _____

substring.

Right Most Sentinal Form Handle Reduce Production

$id1 + id2 * id3$	$id1$	$F \rightarrow id$
$F + id2 * id3$	F	$T \rightarrow F$
$T + id2 * id3$	E	$E \rightarrow T$
$E + id2 * id3$	$id2$	$F \rightarrow id$
$E + F * id3$	F	$T \rightarrow F$
$E + T * id3$	$id3$	$F \rightarrow id$
$E + T * F$	$T * F$	$T \rightarrow T * F$
$E + T$	$E + T$	$E \rightarrow E + T$

$E \Rightarrow$ only see valid rules.

reverse process of top down in rightmost deriv manner.

input top
pushed in
stack

shift,
reduce,
accept and
error

get
elements
from stack
NO:
DATE:

SR Parser :-

Stack	Input	Action
\$	id1 + id2 * id3	Shift
\$ id1	+ id2 * id3 \$	Reduce $F \rightarrow id$
\$ F	"	Reduce $T \rightarrow F$
\$ T	"	Reduce $E \rightarrow T$
\$ E	+ id2 * id3 \$	Shift
\$ E +	id2 * id3 \$	Shift
\$ E + id2	* id3 \$	Reduce $F \rightarrow id$
\$ E + F	"	Reduce $T \rightarrow F$
\$ E + T	"	Shift
\$ E + T *	id3 \$	Shift
\$ E + T * id3	\$	Reduce $F \rightarrow id$
\$ E + T * F	\$	Reduce $T \rightarrow T * F$
\$ E + T	\$	Reduce $E \rightarrow E + T$
\$ E	\$	Accept

Start
variable

id * (id2 + id3)

$E \rightarrow E + T \quad (1)$

$T \rightarrow T * F \quad (3)$

$F \rightarrow id \quad (5)$

NO: _____

DATE: _____

Stack	Input	Action
\$	id1 * (id2 + id3) \$	Shift
\$ id1	* (id2 + id3) \$	Reduce $F \rightarrow id$
\$ F	*(id2 + id3) \$	Reduce $T \rightarrow F$
\$ T	"	Reduce $E \rightarrow T$
\$ E	"	Shift
\$ E *	"	Shift
\$ E * (	id2 + id3) \$	Shift
\$ E * (id2	+ id3) \$	Reduce $F \rightarrow id$
\$ E * (F	+ id3) \$	Reduce $T \rightarrow id$
\$ E * (T	+ id3) \$	Shift
\$ E * (T +	id3) \$	Shift
\$ E * (T + id3	) \$	Reduce $F \rightarrow id$
\$ E * (T + F	) \$	Shift

Bison:

NO:

DATE: 8 May 25

tokenization → scan.l

↓ lex file

↓ three components

[a-zA-Z_][

a-zA-Z0-9_]*

{ printf("<ID,

%s>\n", yytext);

return ID;

/* option

noyywrap.

%{

// definition %}

%%

// RE { }

%{

int main() {

yylex();

return 1;

⇒ flex scan.l

#include <stdio.h>

gcc lex.yy.c -o scanner -lfl

#include

"parser.h"

/* scanner

num1

↳ <ID, num1>

• // for any character

{ printf("< Error, %s>",

yytext);

/* ignoring error */

NO:

parser.y

DATE:

```

%{ #include <stdio.h>
   int yylex (void); void yyerror();
%}
/* token ID          built in          const char *
   string
*/
/* start program

```

```

%}

```

```

program : /* empty */ [ ] ;

```

```

%}

```

```

int main() { yyparse(); return 0; }

```

```

void yyerror(const char * string) {
    printf ("Error : %s", string);
}

```

⇒ bison -d parser.y

⇒ ls , -lex scan.l , gcc lex.yy.c
 parser.tab.c
 compiler -lfl